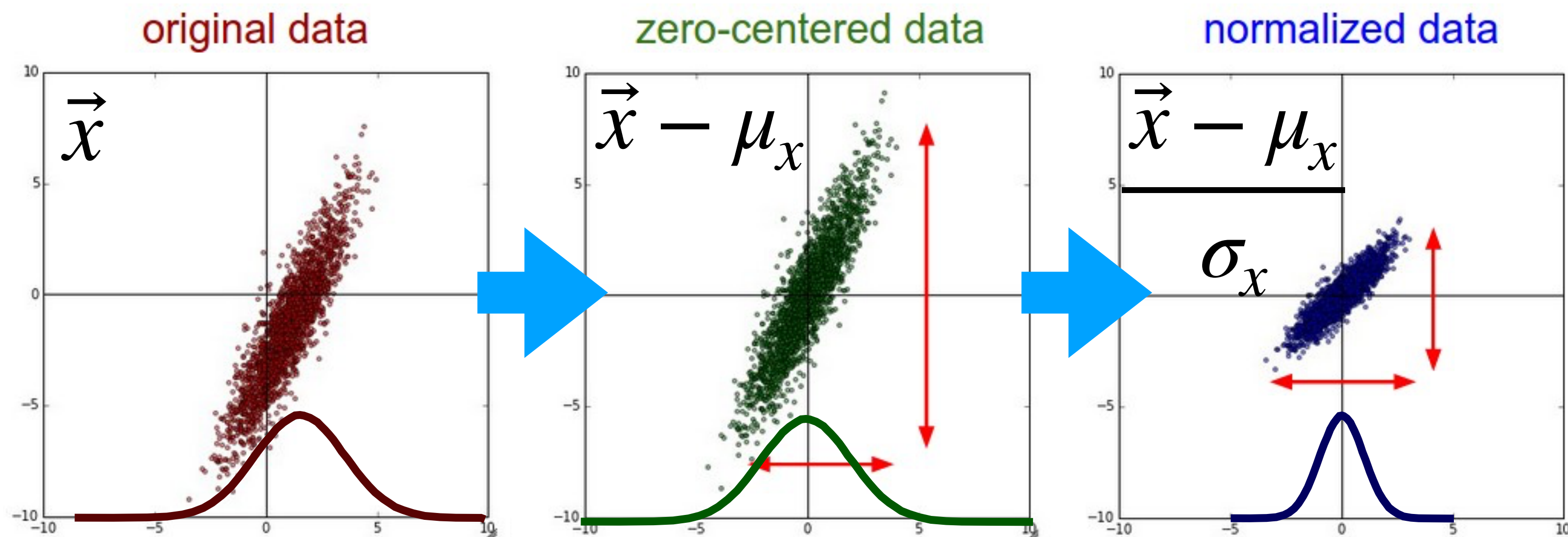




Lecture 4: Training in Practice

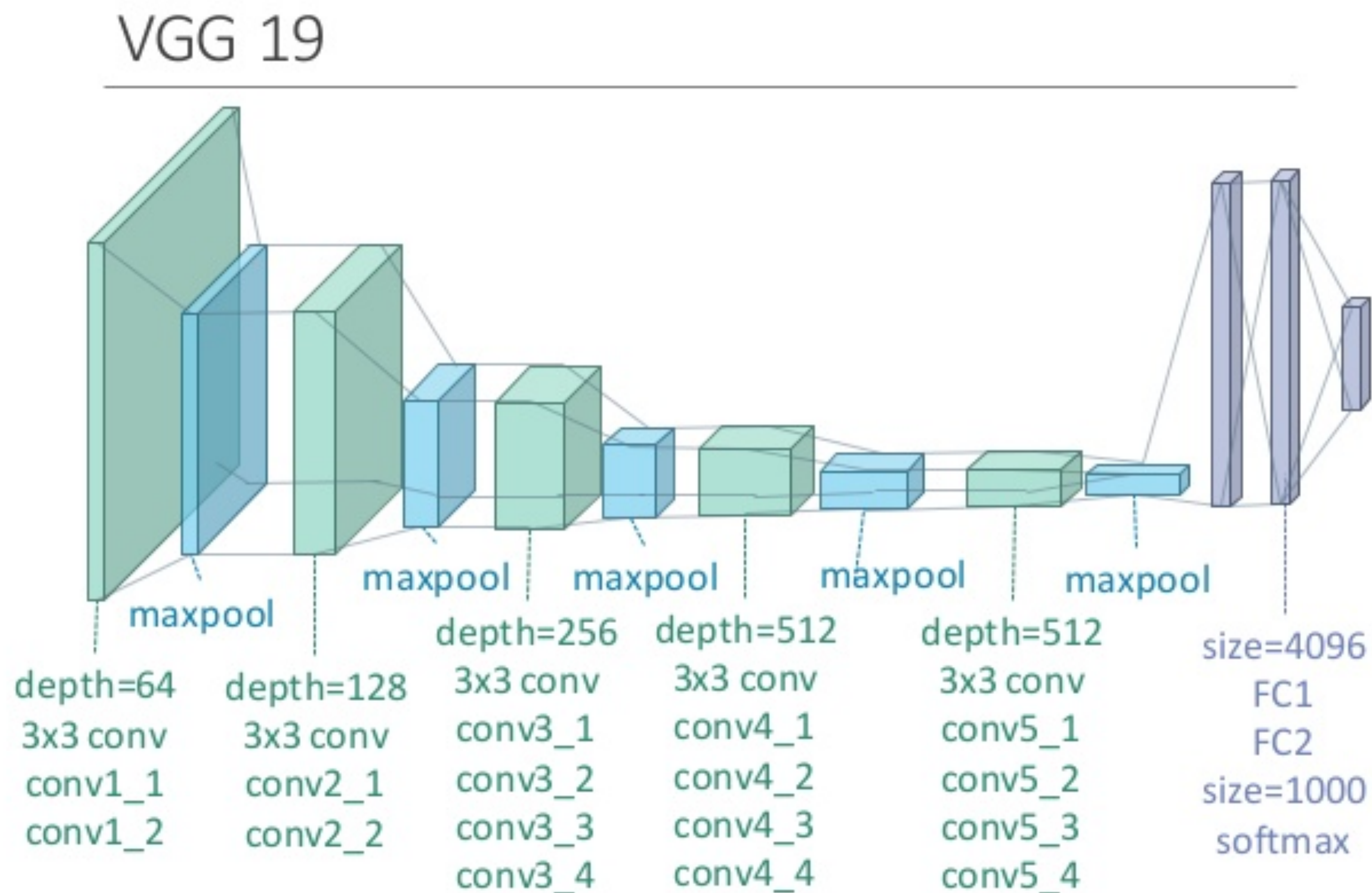
Batch Normalization

- It is good practice to give normalized inputs to a layer
 - With inputs having the similar magnitude, all weights are equal important in the gradient
 - Prevents explosion of the loss function
- This can be done automatically with BatchNormalization
 - non-learnable shift and scale parameters, adjusted batch by batch
 - When this is used, bias is not needed (you might have noticed that CNNs have no bias parameters)



The full network

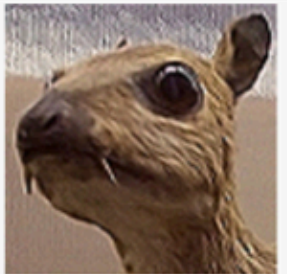
- A full ConvNN is a sequence of Con2D+Pooling (+BatchNormalization +Dropout) layers
- The Conv+Pooling layer reduces the 2D image representation
- The use of multiple filters on the image makes the output grow on the third dimension (channels)
- One can flatten the output at some point and pass the result to a dense layer



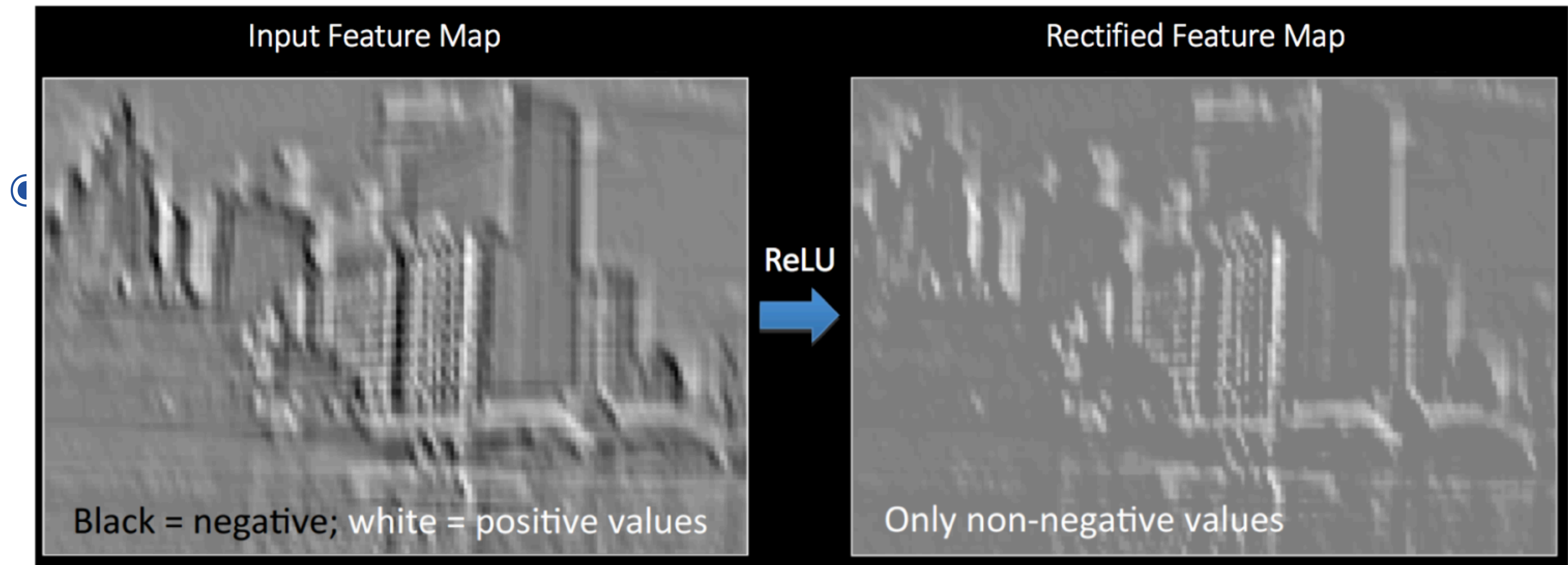
CERN What does a ConvNN learn?

- Each filter alters the image in a different way, picking up different aspects of the image
- edges oriented in various ways
- enhancing / blur of certain features
- It is interesting to check what each filter is doing

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

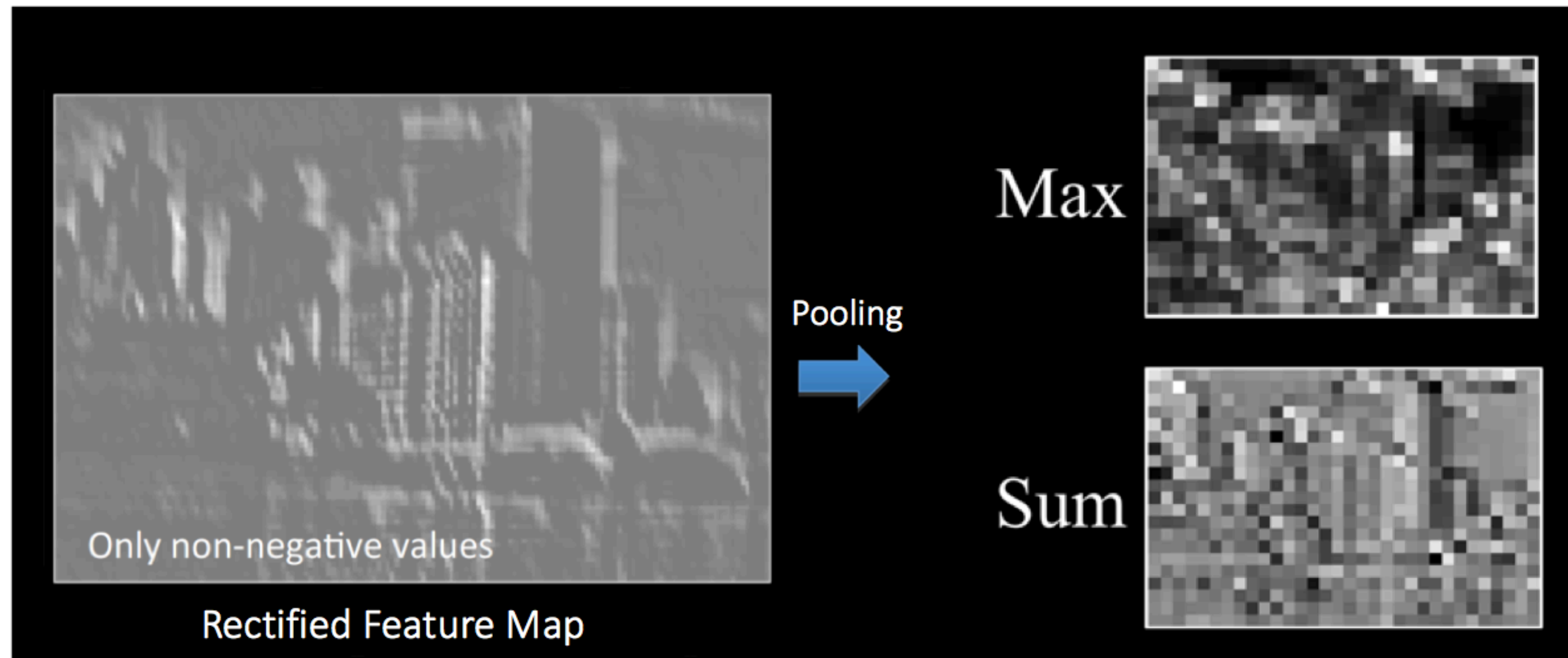
Operation	Filter	Convolved Image
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

What does a ConvNN learn?



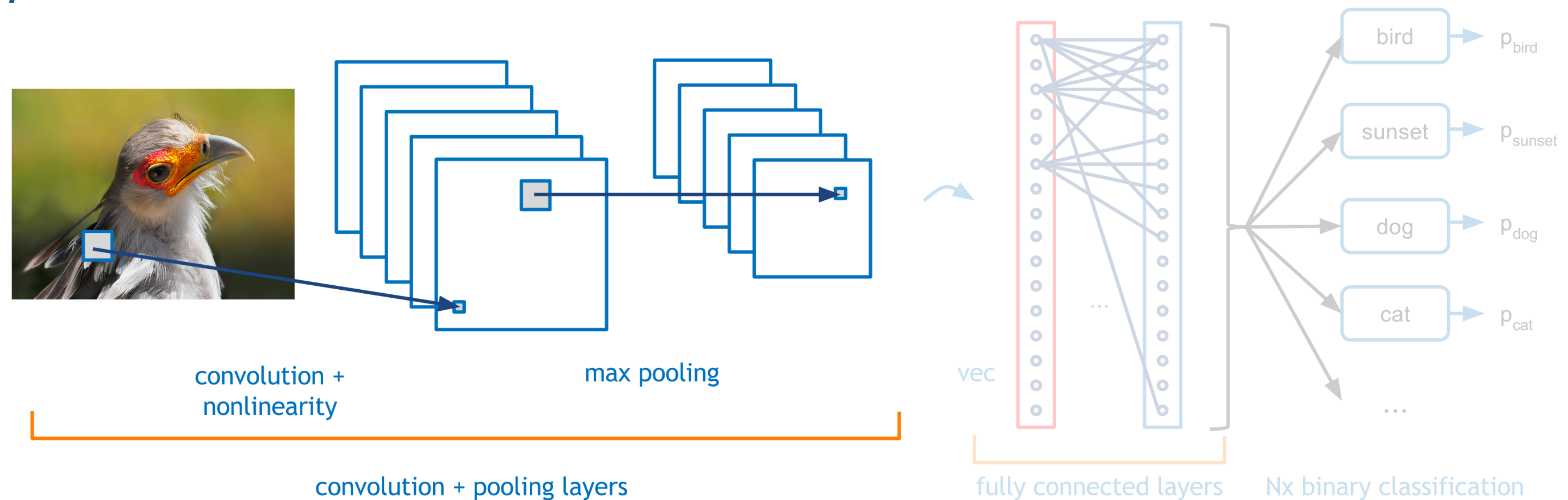
What does a ConvNN learn?

- *Pooling is important in smoothing out the image*
- *reduce parameters downstream (and prevent overfitting)*
- *makes processing independent to local features (distortion, translation)*
- *yields a scale invariant representation of the image (which could be good or bad)*



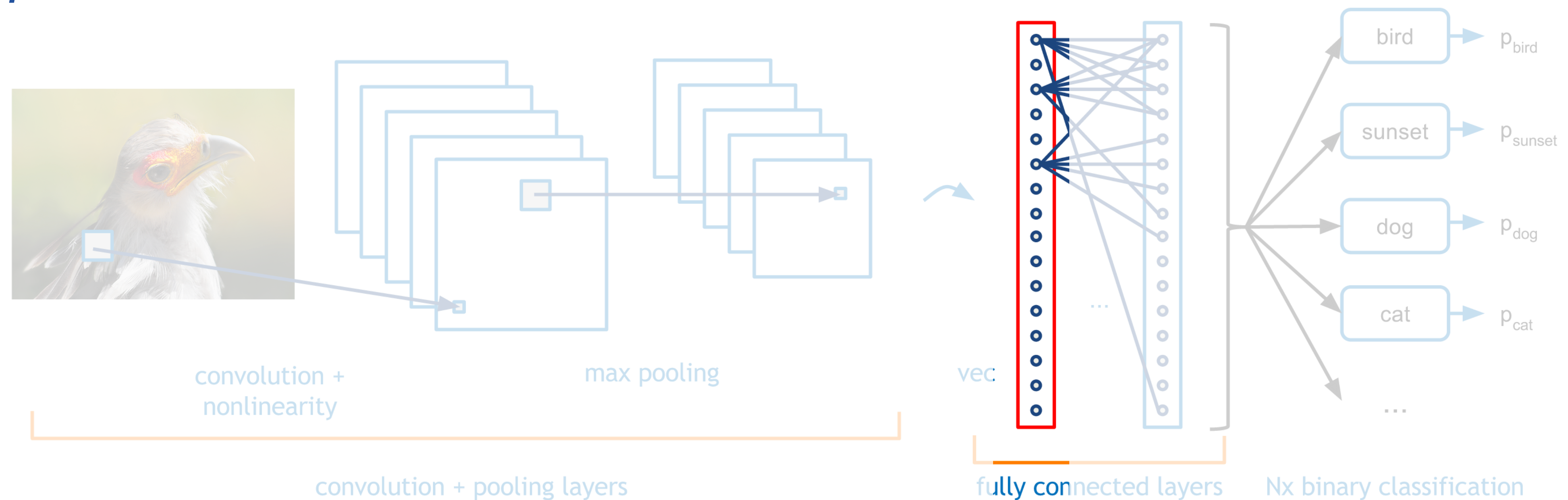
Two tasks in one network

- *The Conv layer starts from RAW data and defines interesting quantities (high level features, HLF)*
- *The HLFs replace the physics-motivated inputs of a DNN*
- *The DNN at the end exploits the engineered features to accomplish the task*



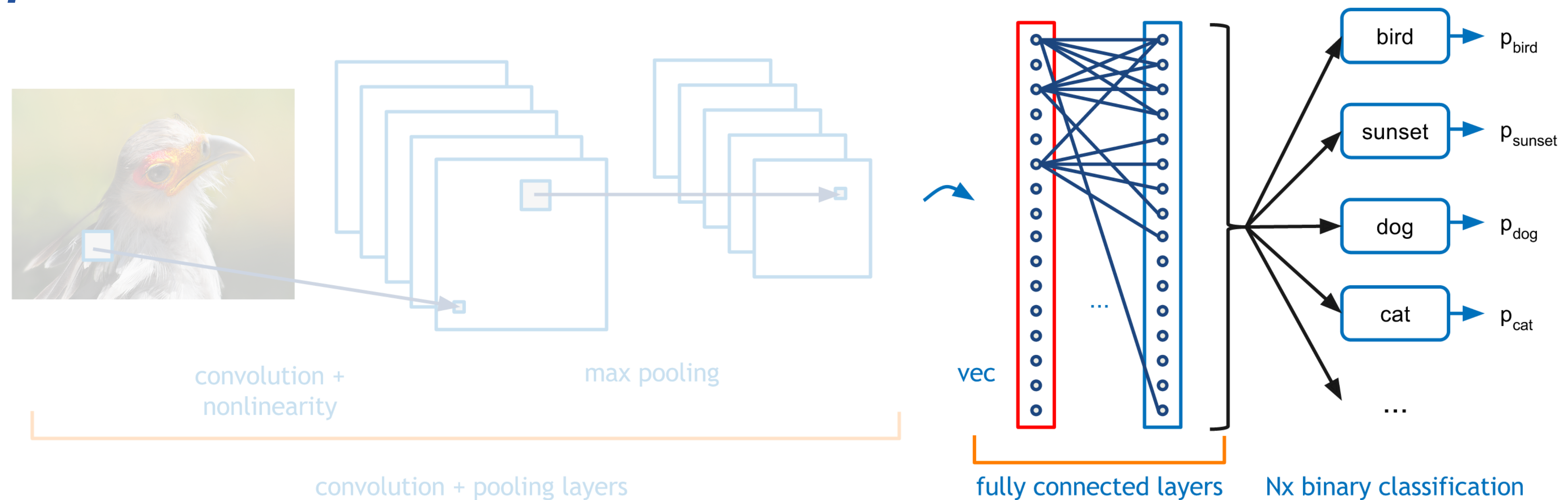
Two tasks in one network

- *The Conv layer starts from RAW data and defines interesting quantities (high level features, HLF)*
- *The HLFs replace the physics-motivated inputs of a DNN*
- *The DNN at the end exploits the engineered features to accomplish the task*



Two tasks in one network

- *The Conv layer starts from RAW data and defines interesting quantities (high level features, HLF)*
- *The HLFs replace the physics-motivated inputs of a DNN*
- *The DNN at the end exploits the engineered features to accomplish the task*



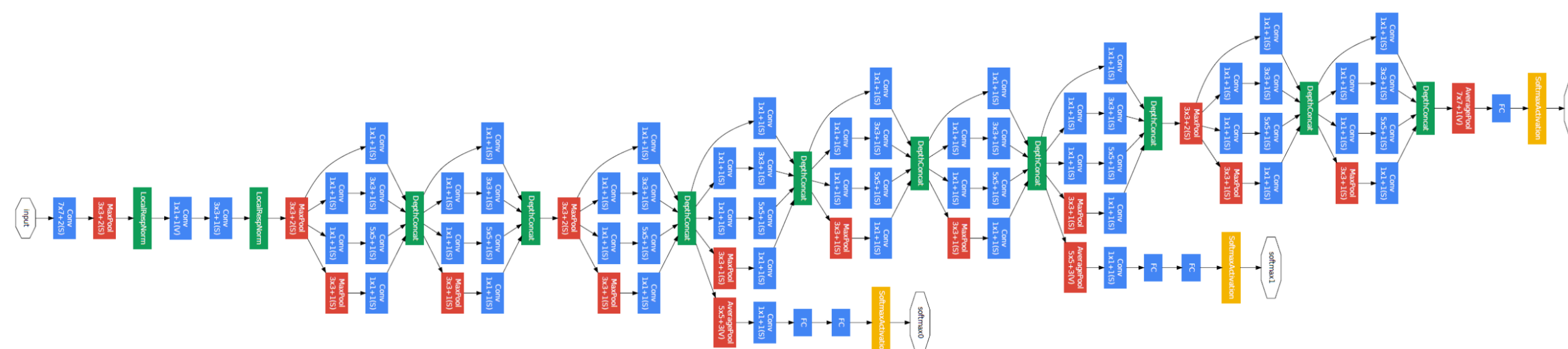
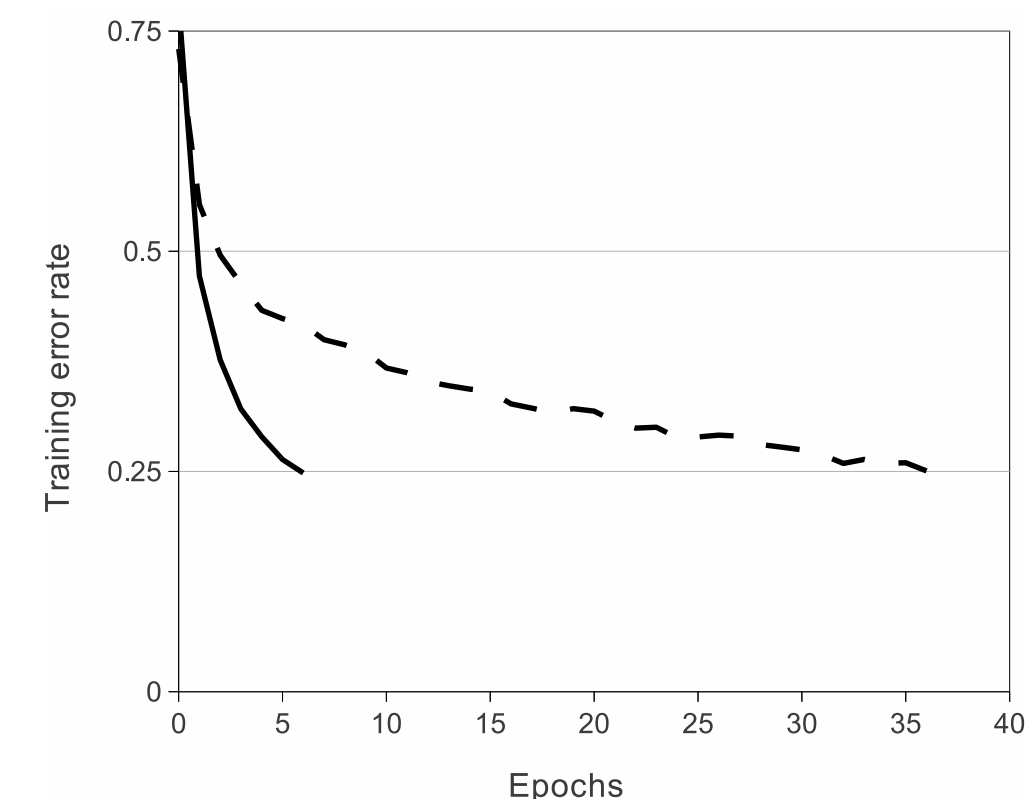
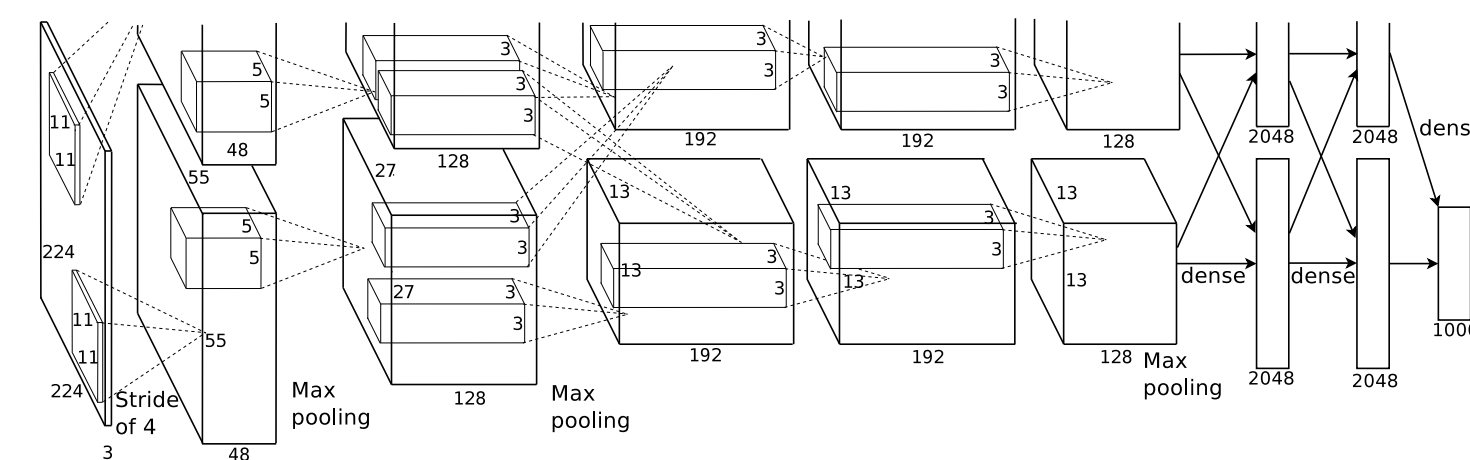
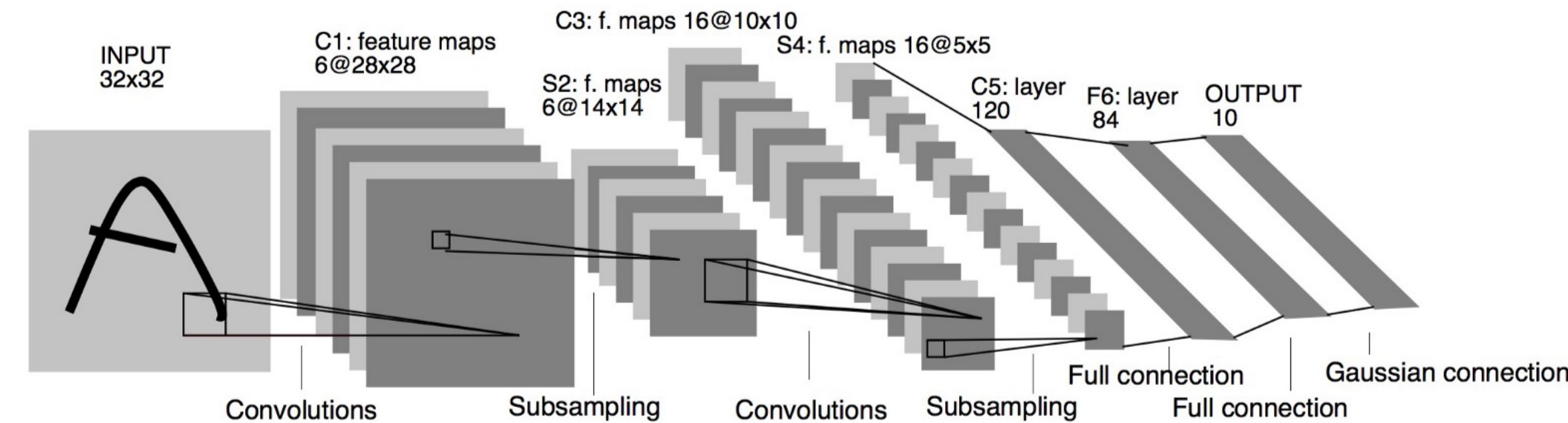
A history of ConvNns

● Neocognitron (1980): translation-invariant image processing with NNs

● LeNet (1989): the very first ConvNN, designer for digit recognition (ZIP codes)

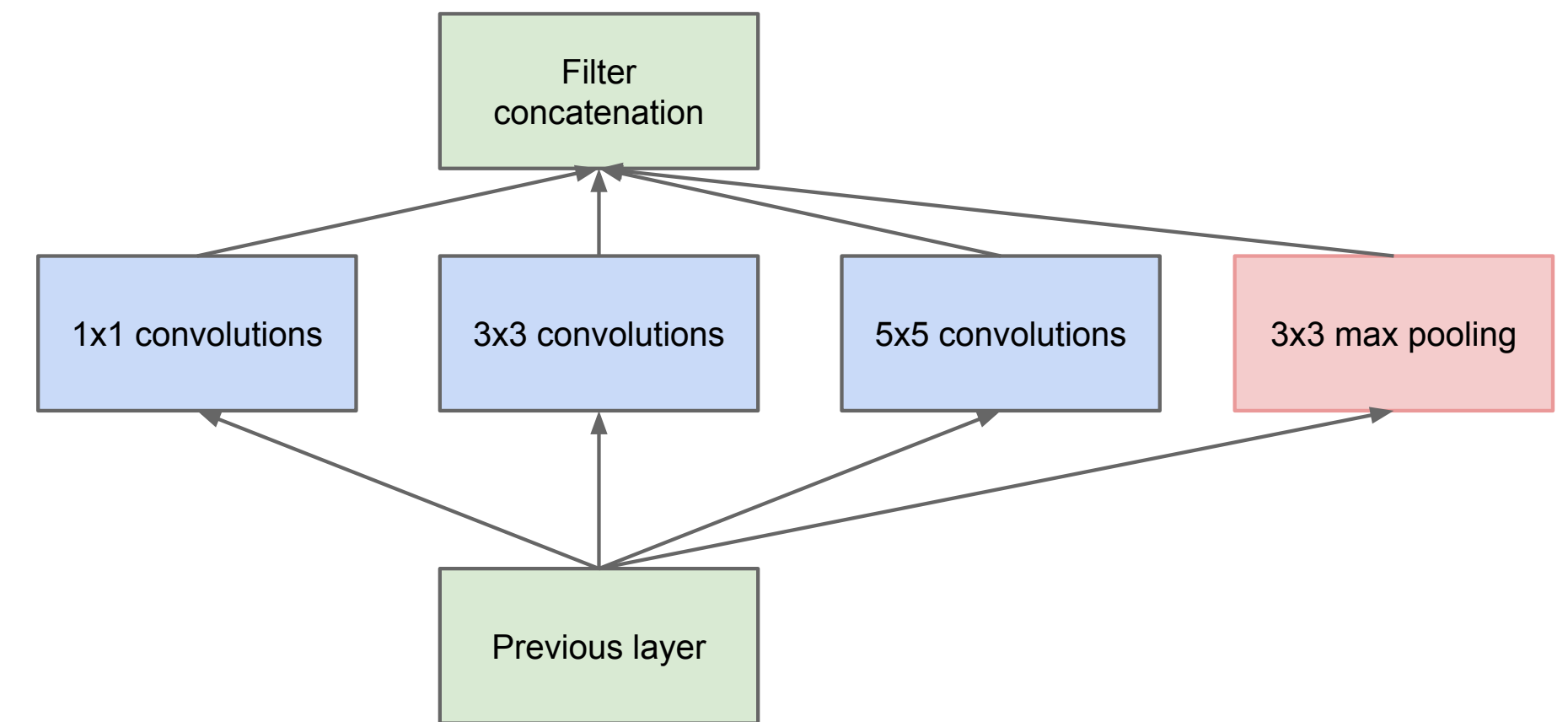
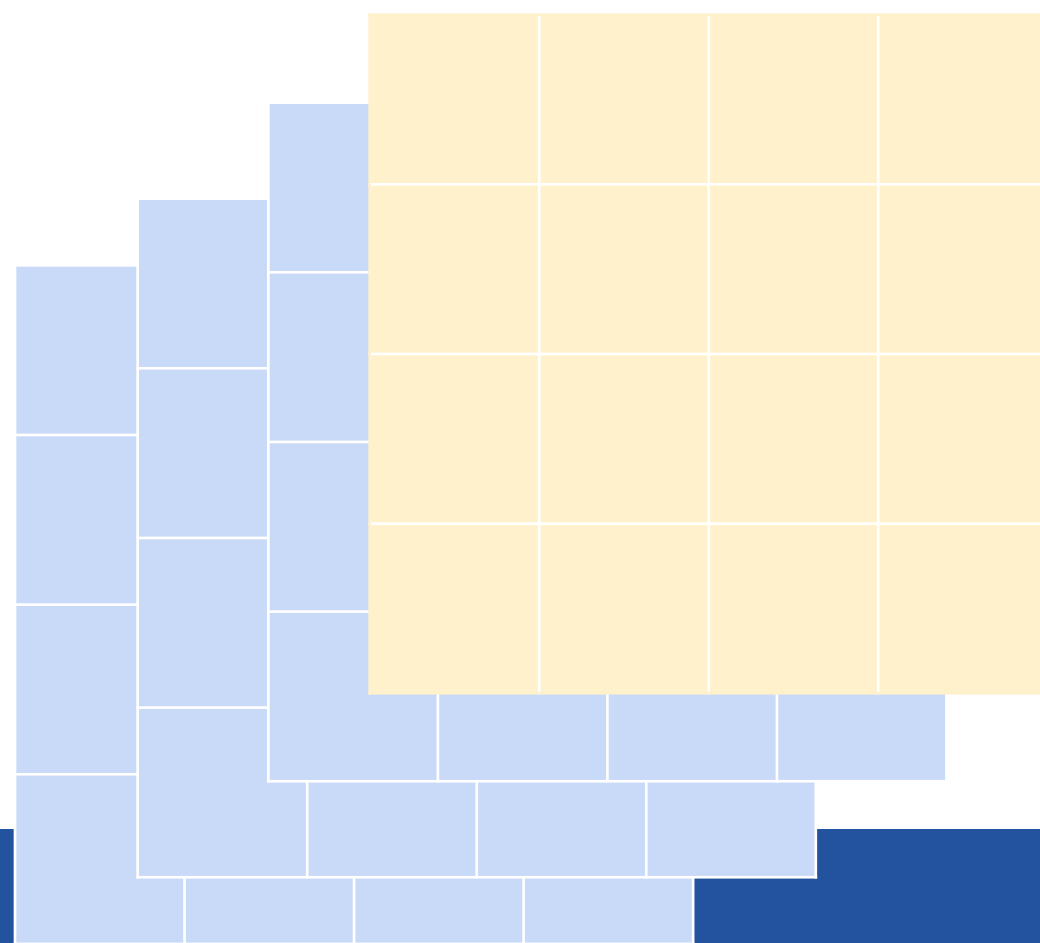
● AlexNet (2012): the first big ConvNN (60M parameters, 650K nodes), setting the stage of the art: trained on GPUs, using ReLU and Dropout

● GoogleNet (2014): built on AlexNet, introduced an inception model to reduce the number of parameters

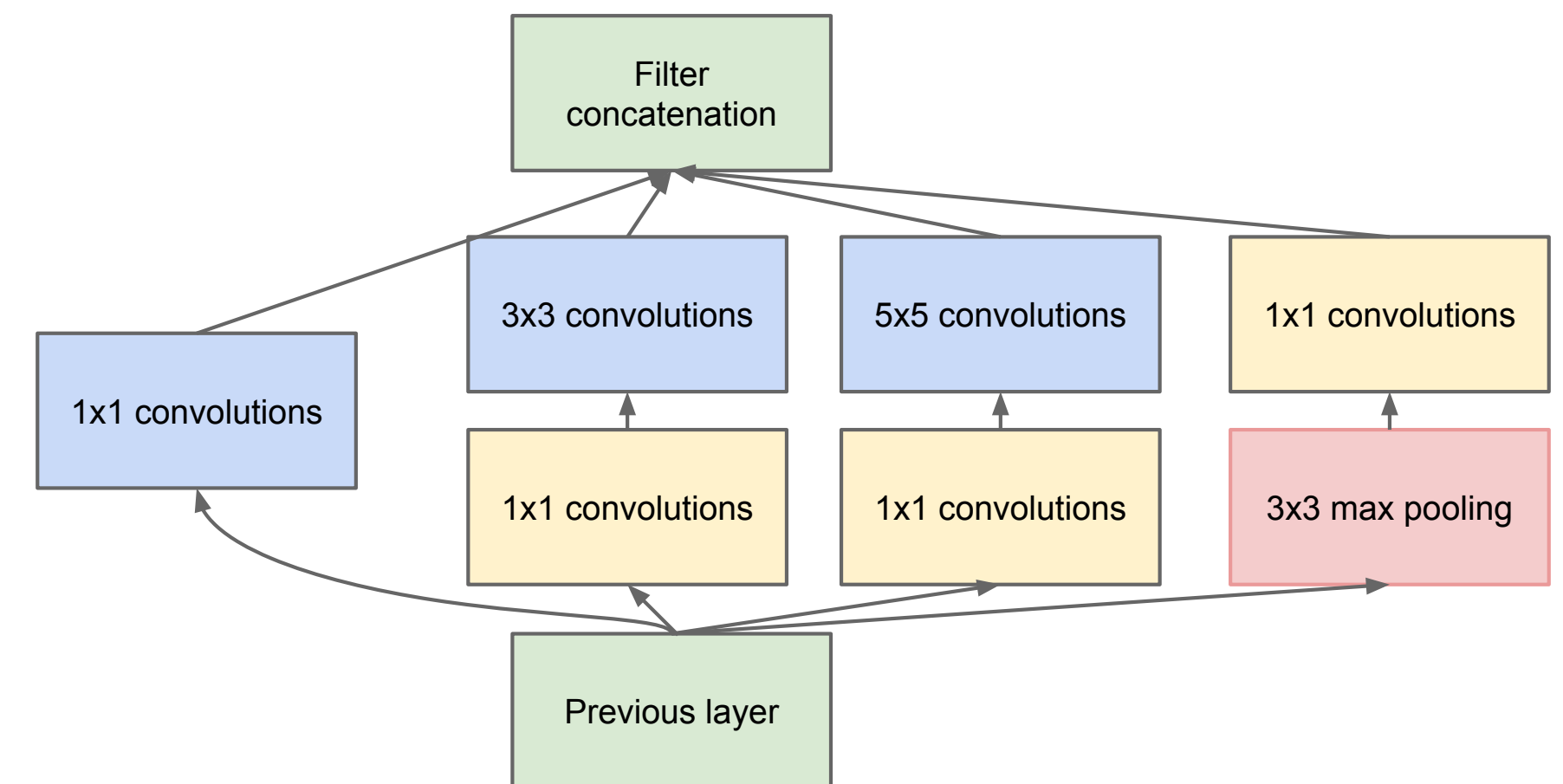


Inception Module

- ⦿ *Rather than going deeper and deeper, inception architectures go wider*
- ⦿ *Several conv layers, with different filter size, process the same inputs*
- ⦿ *This way, more features can be detected from the same image*
- ⦿ *The outcome of this parallel processing is then recombined through a concatenation step as channels of an image*



(a) Inception module, naïve version

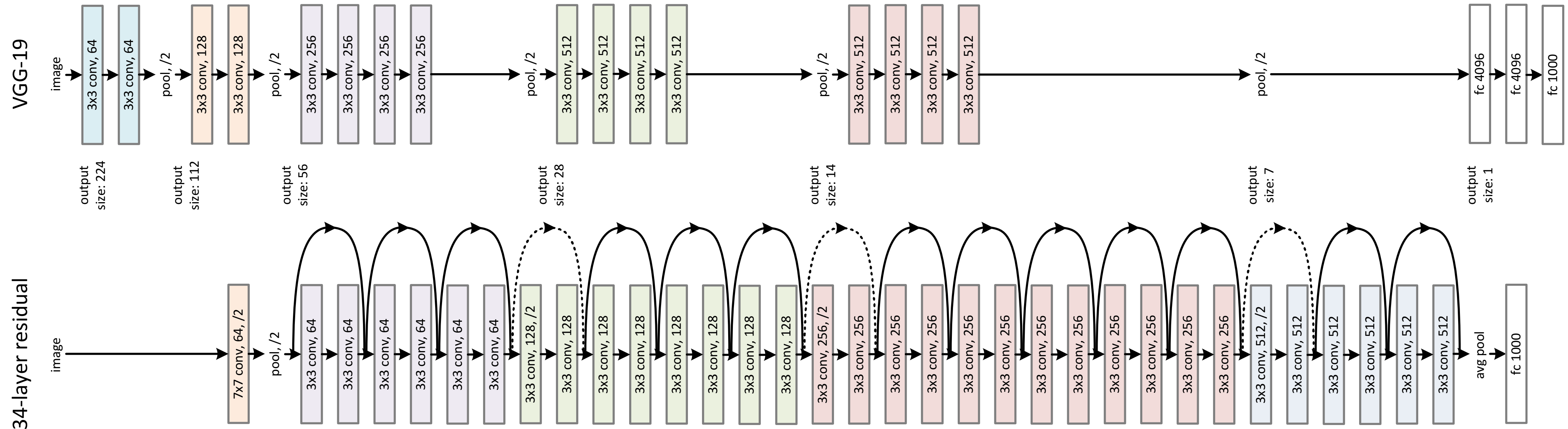
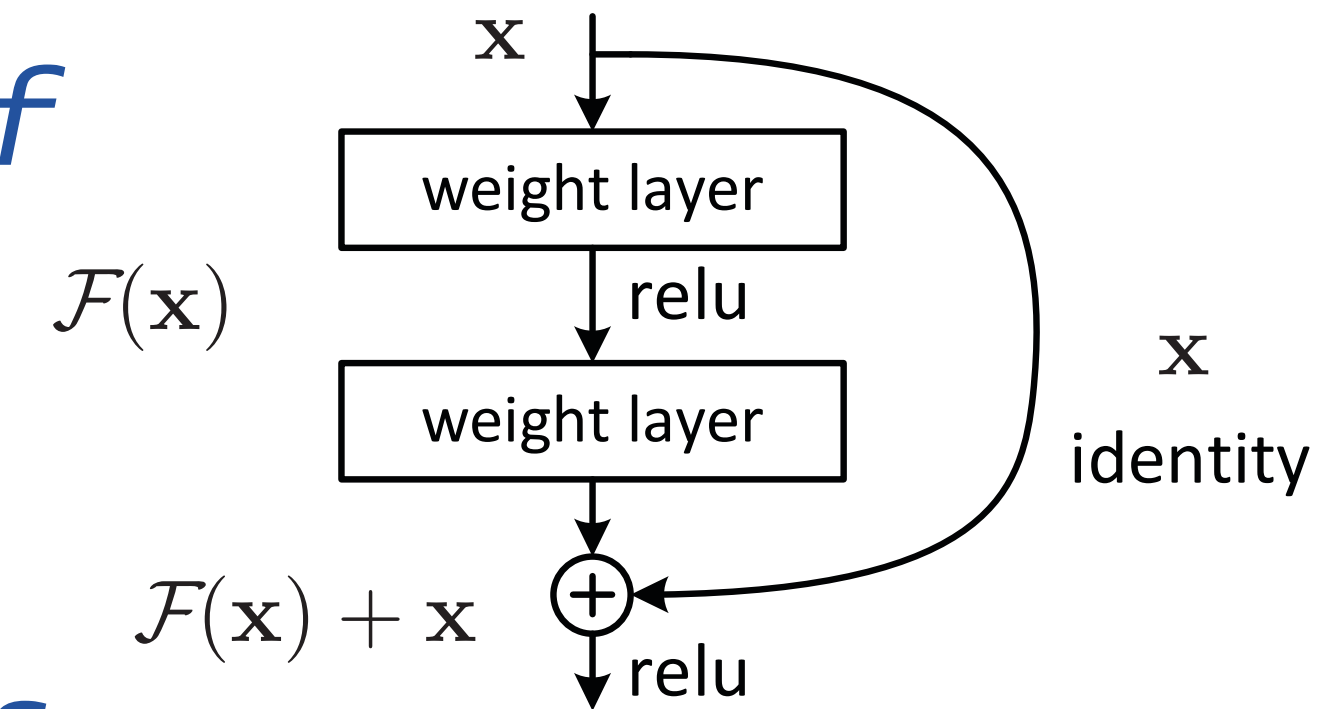


(b) Inception module with dimension reductions

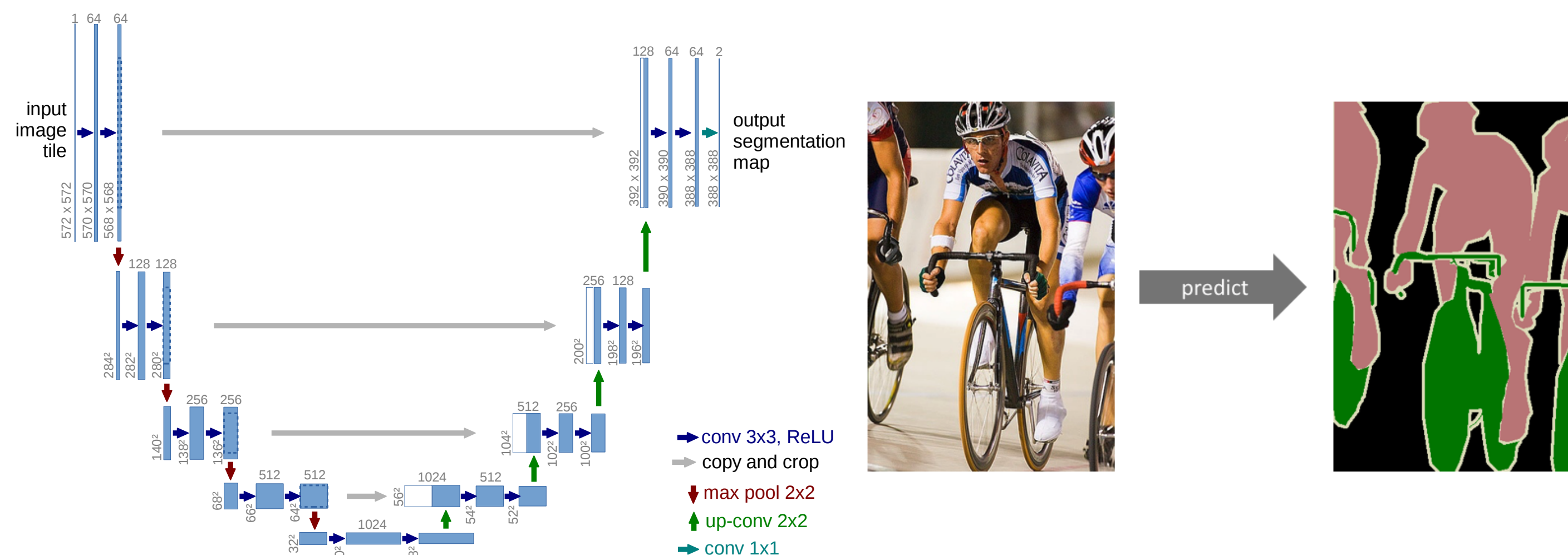
A history of ConvNns

● VGGNet(2014): exploit small filters (3x3, 1x1), previously considered not optimal in a stack of Conv layers

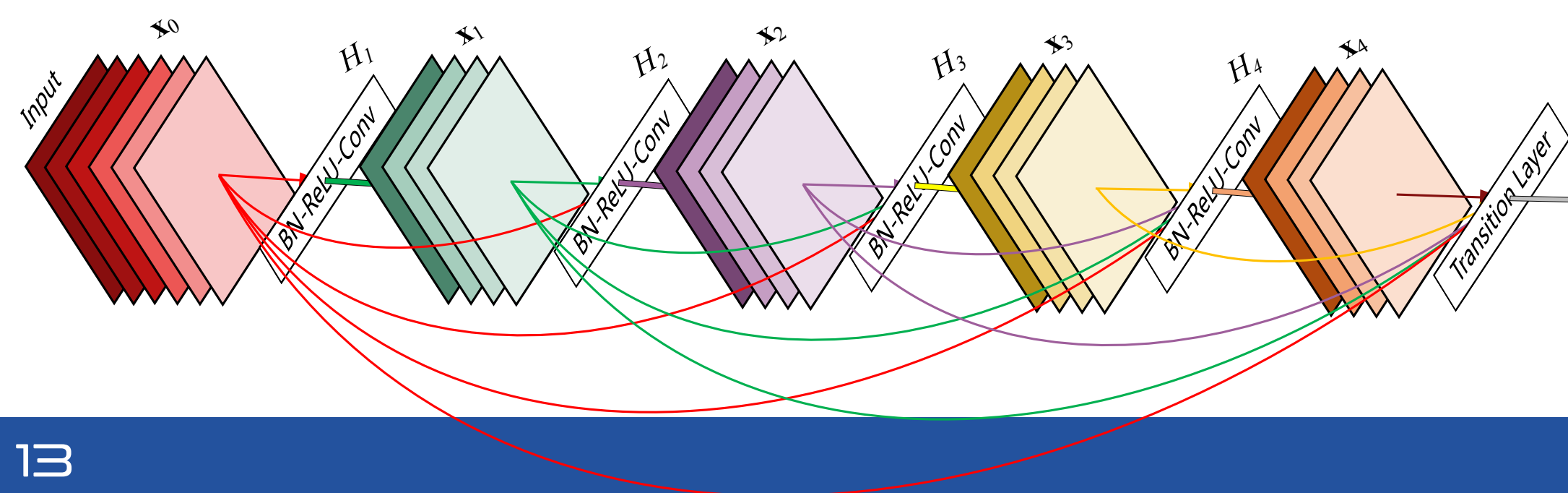
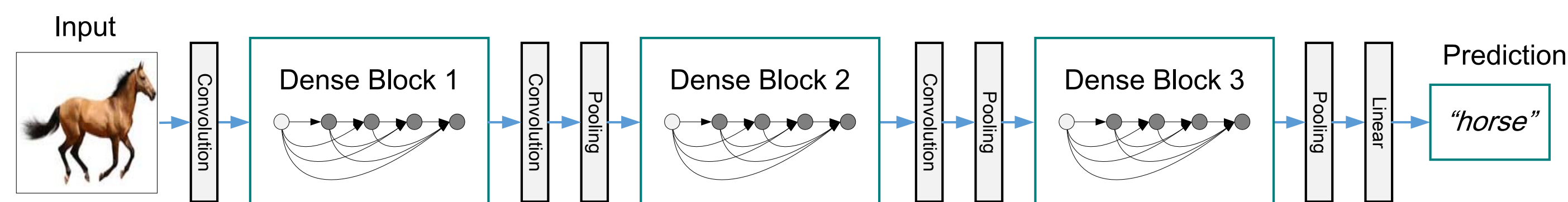
● ResNet (2015): implemented skip connections, which allows to focus the learning on residuals (difference between inputs and outputs)



- U-net (2015): conv layers with skip connections, in a downsampling+upsampling U-shaped sequence. Introduced for semantic segmentation



- DenseNet (2016): uses skip connections between a given layer and all the layers downstream in a dense block, with Conv layers in between

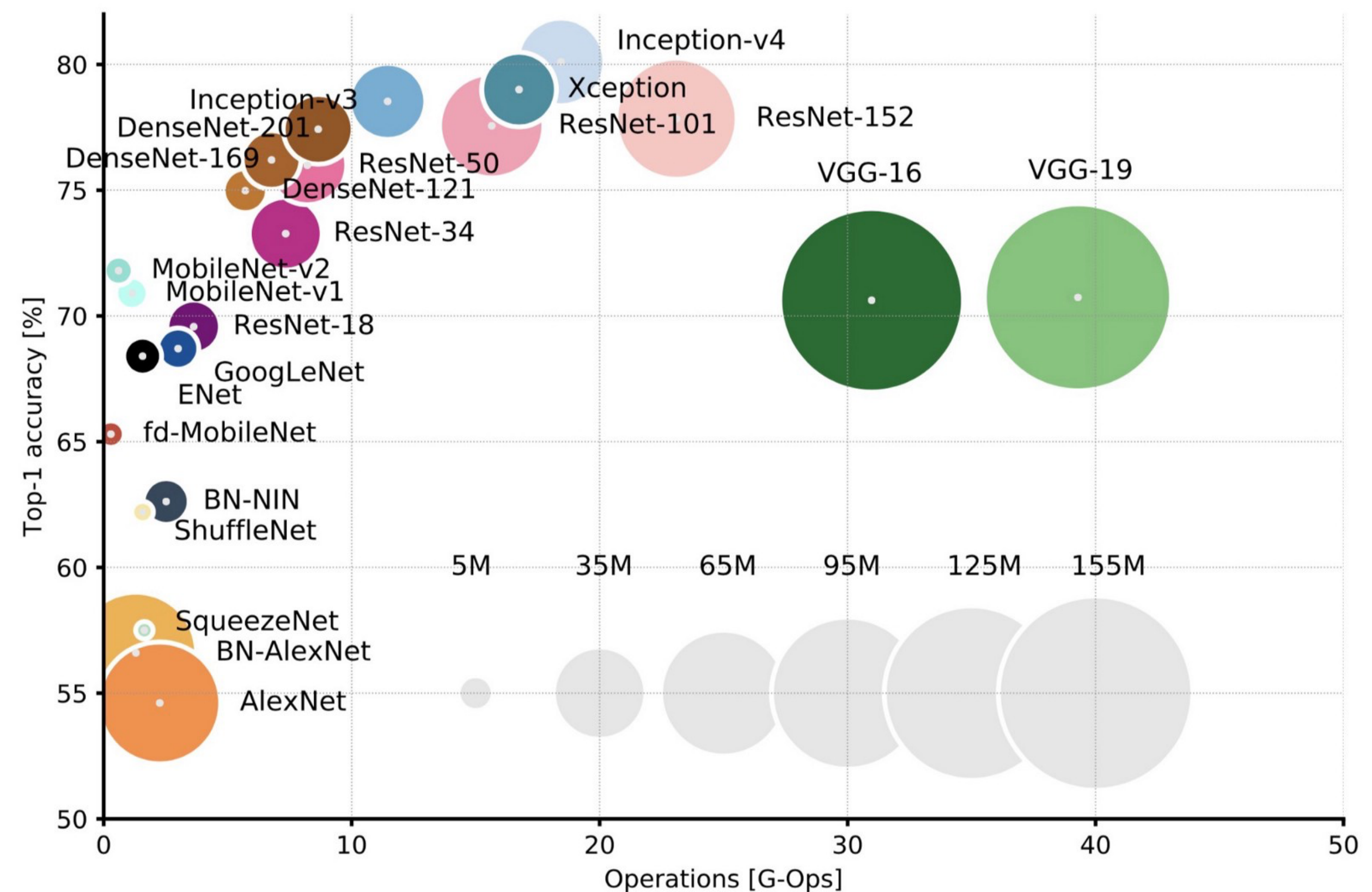


The computational cost

- In this evolution, computing-vision networks drastically improved in performance
- This came at a big cost in complexity (number of parameters and operations)
- The inference with this network became particularly slow
- Many cloud providers provide optimized versions of these networks:

 - you can just use them (re-training if needed), rather than inventing your own one

<https://towardsdatascience.com/neural-network-architectures-156e5bad51ba>

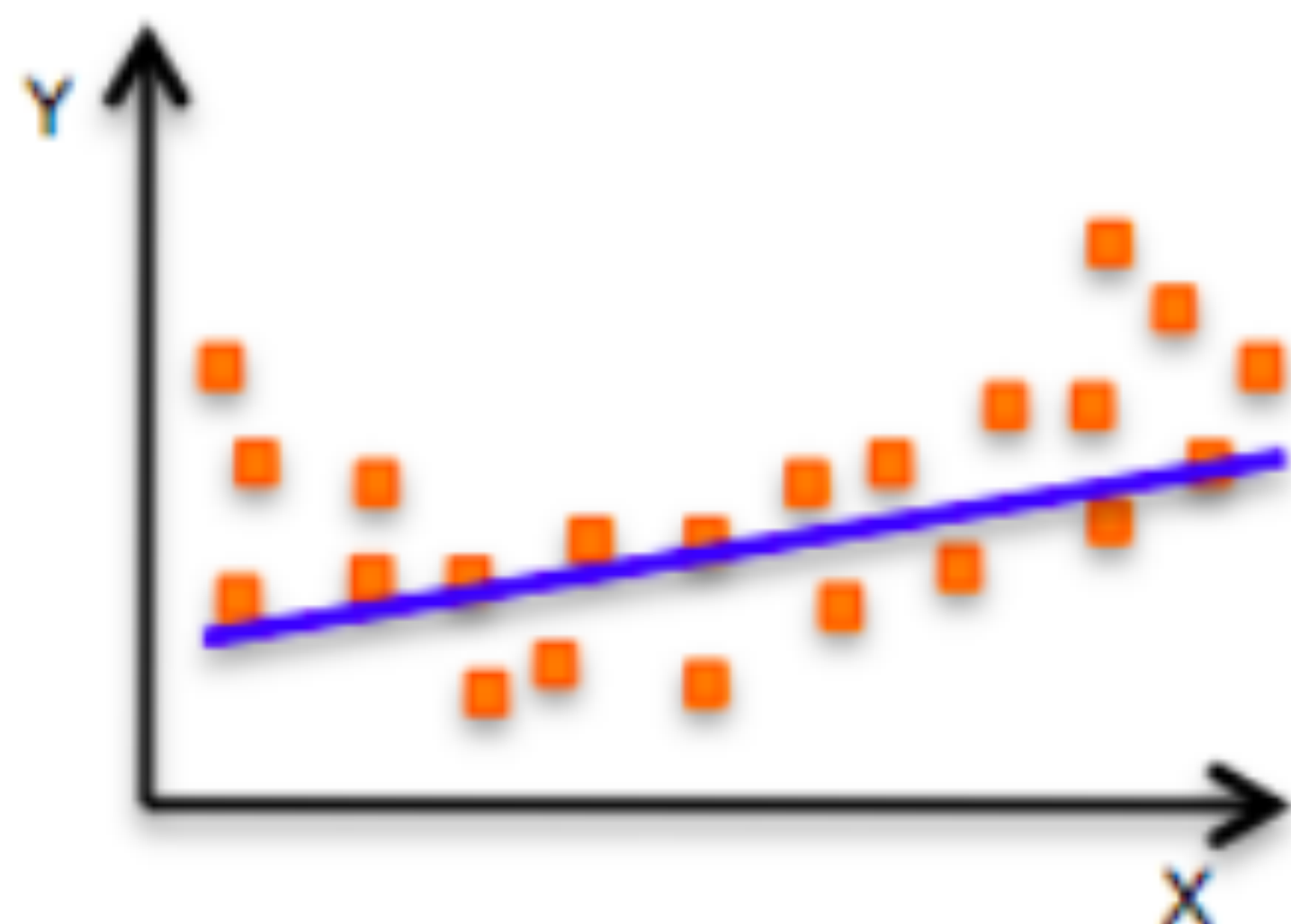


Program for Today

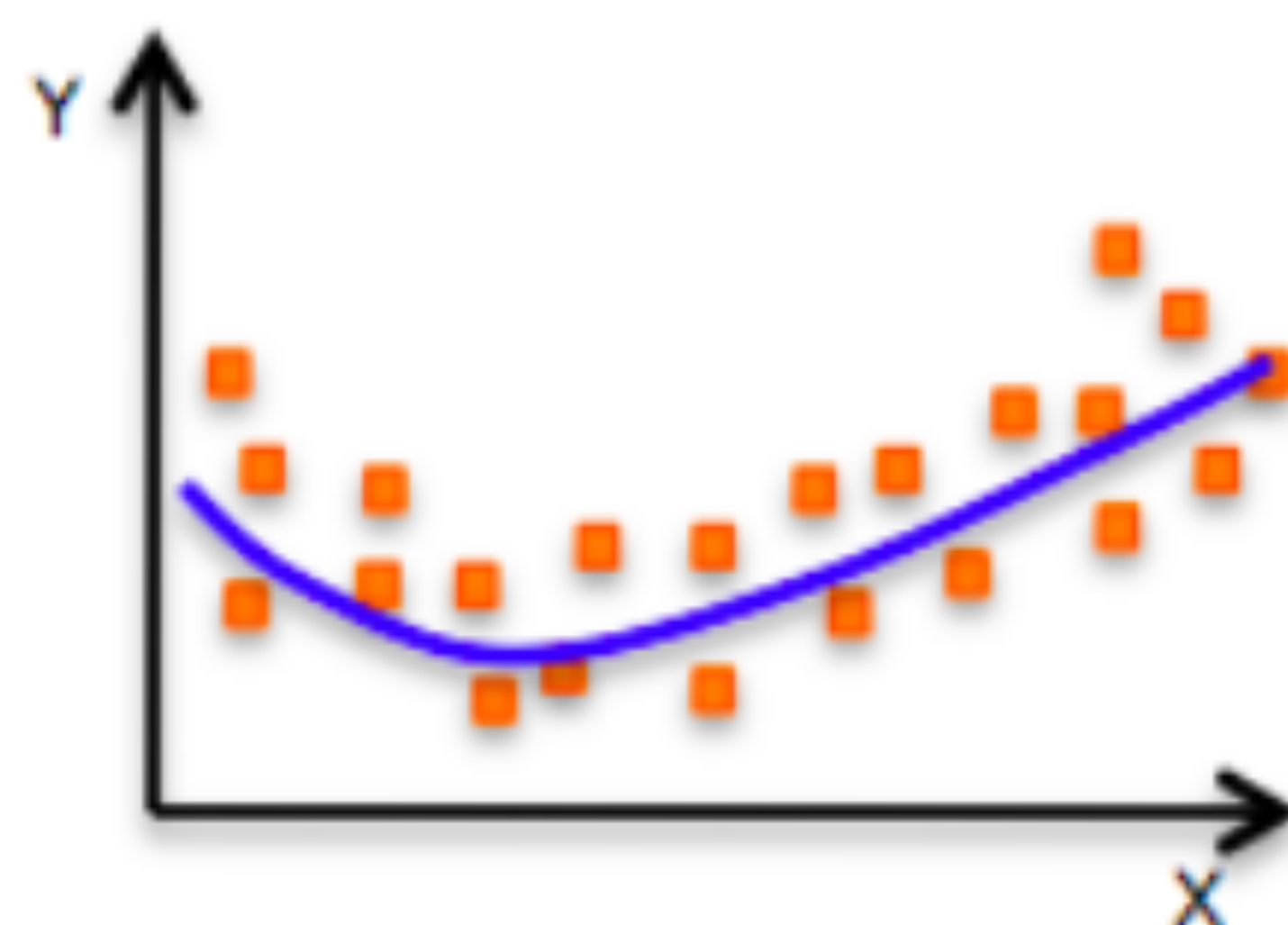
- [Table of Contents](#)
- [Acknowledgements](#)
- [Notation](#)
- ✓ • [1 Introduction](#)
- [Part I: Applied Math and Machine Learning Basics](#)
 - ✓ ○ [2 Linear Algebra](#)
 - ✓ ○ [3 Probability and Information Theory](#)
 - ✓ ○ [4 Numerical Computation](#)
 - ✓ ○ [5 Machine Learning Basics](#)
- [Part II: Modern Practical Deep Networks](#)
 - ✓ ○ [6 Deep Feedforward Networks](#)
 - ✓ ○ [7 Regularization for Deep Learning](#)
 - ✓ ○ [8 Optimization for Training Deep Models](#)
 - ✓ ○ [9 Convolutional Networks](#)
 - [10 Sequence Modeling: Recurrent and Recursive Nets](#)
 - ✓ ○ [11 Practical Methodology](#)
 - [12 Applications](#)
- [Part III: Deep Learning Research](#)
 - [13 Linear Factor Models](#)
 - [14 Autoencoders](#)
 - [15 Representation Learning](#)
 - [16 Structured Probabilistic Models for Deep Learning](#)
 - [17 Monte Carlo Methods](#)
 - [18 Confronting the Partition Function](#)
 - [19 Approximate Inference](#)
 - [20 Deep Generative Models](#)
- [Bibliography](#)
- [Index](#)

	Topic	Tutorial
Sep 15	Intro & class description + Basic of machine learning + Dense NN	
Sep 22	Probability Theory + ML Training	
Sep 29	Statistics + Convolutional NN	<i>Basic jupyter + DNN on mnist</i>
Oct 6	Training in practice: regularization, optimization, etc	<i>Convolutional NNs with MNIST</i>
Oct 13	Unsupervised learning and anomaly detection	<i>Autoencoders with MNIST</i>
Oct 20	Graph NNs	<i>Tutorial on Graph NNs</i>
Oct 27	Transformers	
Nov 3	Network compression (pruning, quantization, Knowledge Distillation)	
Nov 10	Tutorial on hls4ml	
Nov 17	Generative models: GANs, VAEs, etc	<i>GAN and VAE</i>
Nov 24	Reinforcement Learning	
Dec 1	Normalizing flows	<i>Normalizing flows</i>
Dec 8	Quantum Machine Learning (tbc)	
Dec 15	Exams To Be Confirmed	

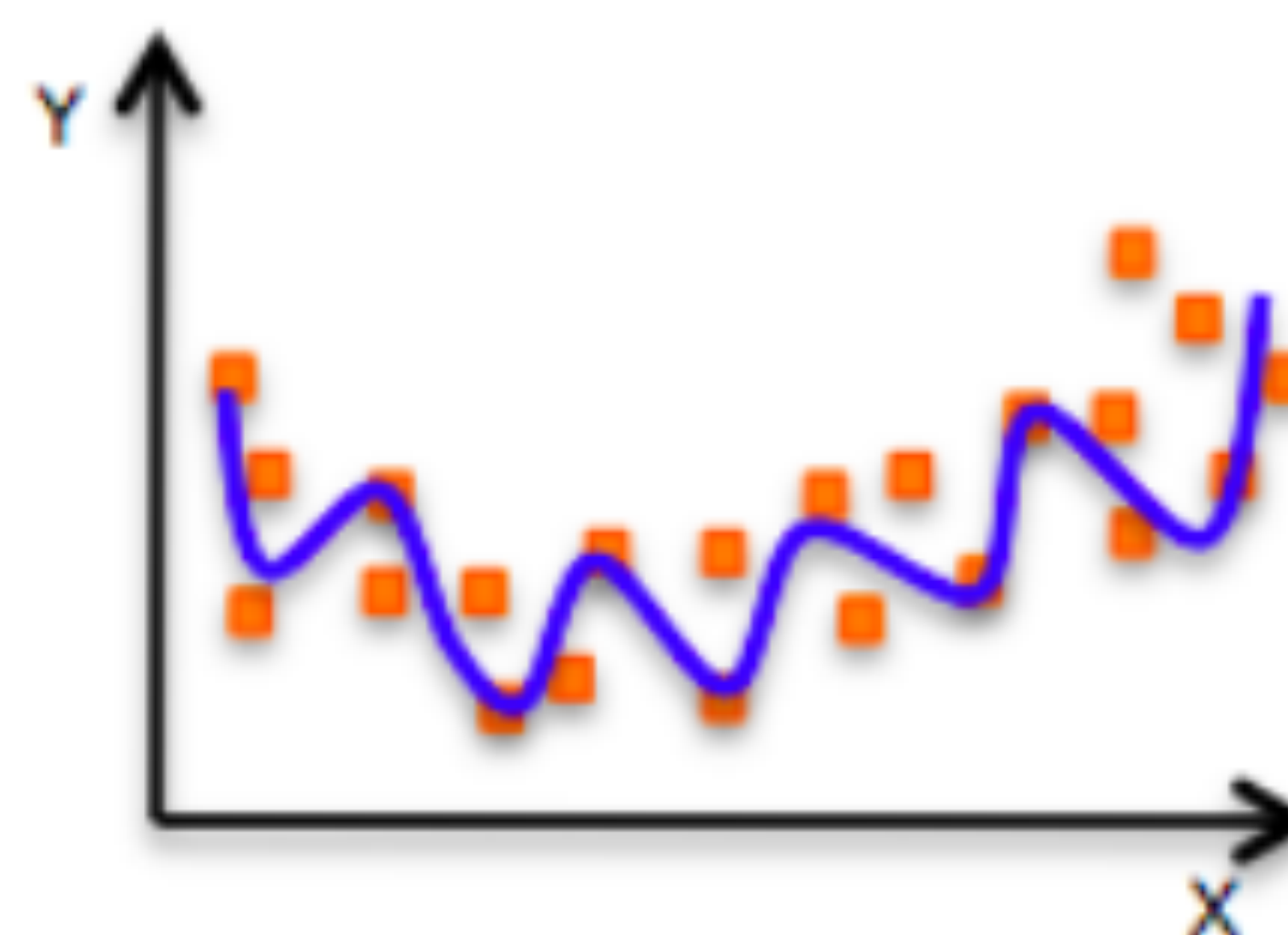
Regularisation



Underfitting



Just right!



overfitting

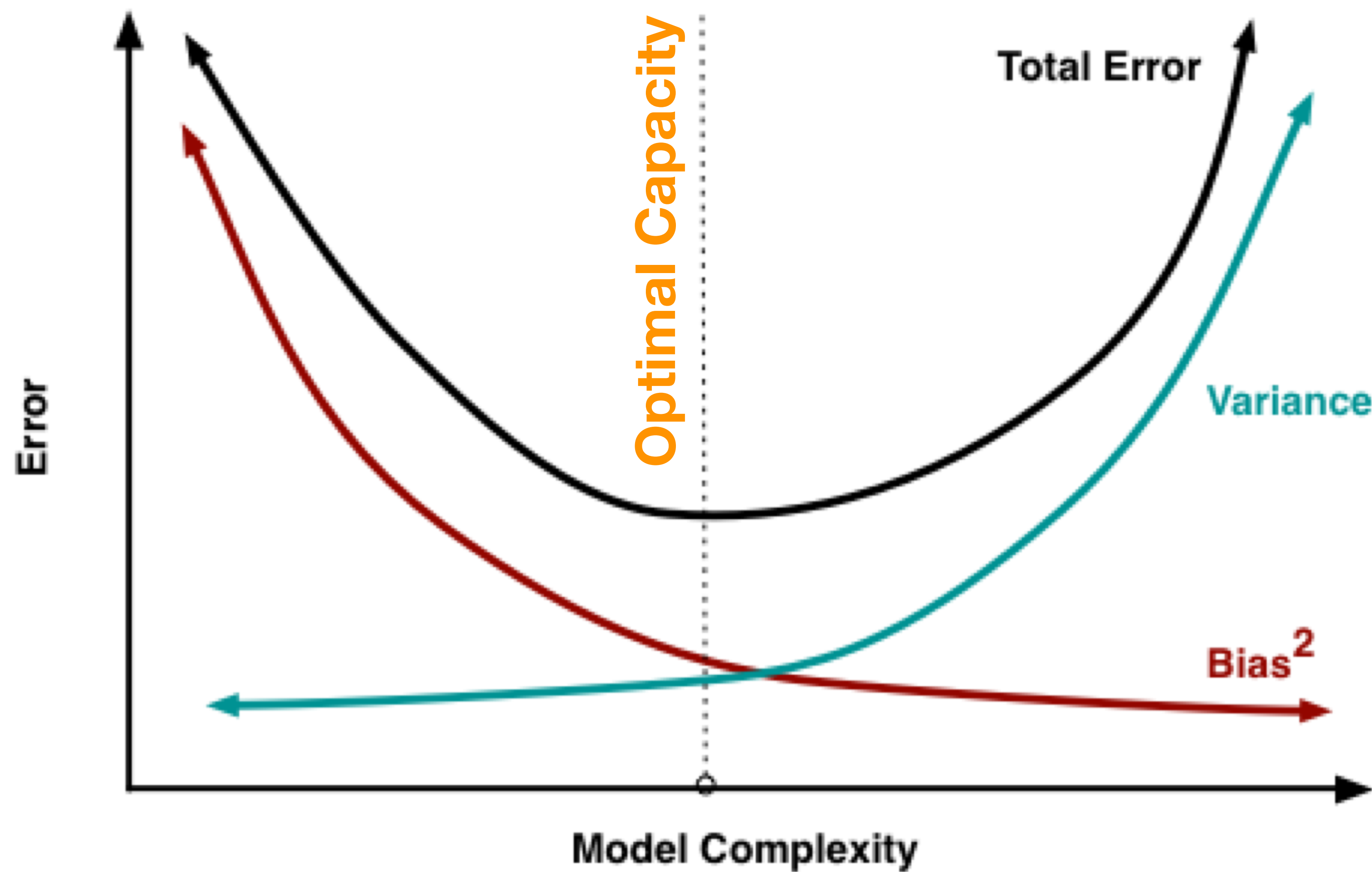
Bias vs Variance

Generalization error can be written as the sum of three terms:

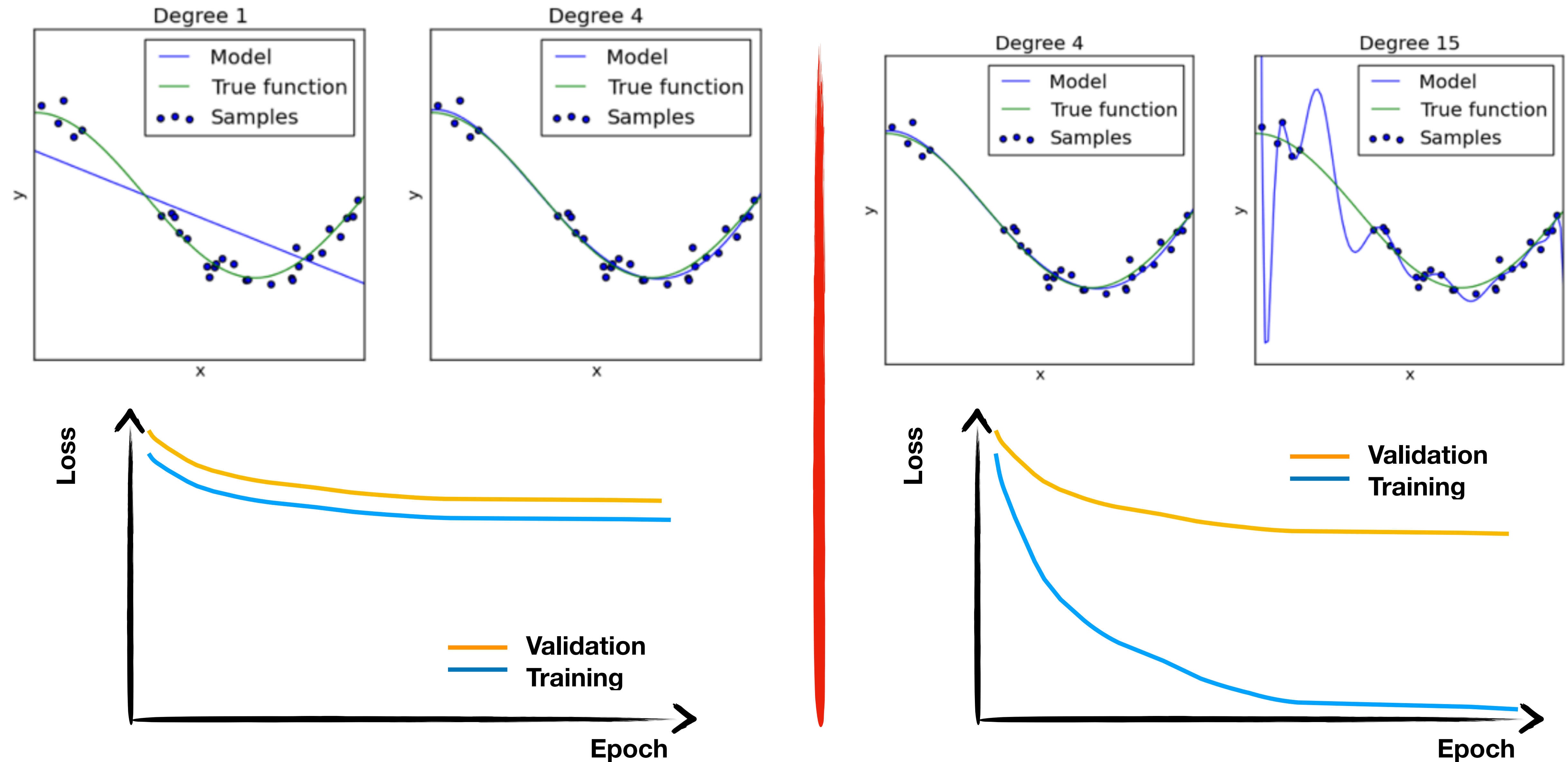
- The *intrinsic statistical noise* in the data
- the *bias* wrt the mean
- the *variance* of the prediction around the mean

$$E[(y - h(x))^2] = \underbrace{E[(y - \bar{y})^2]}_{\text{Noise}} + \underbrace{(\bar{y} - \bar{h}(x))^2}_{\text{Bias Squared}} + \underbrace{E[(h(x) - \bar{h}(x))^2]}_{\text{Variance}}$$

Bias vs Variance



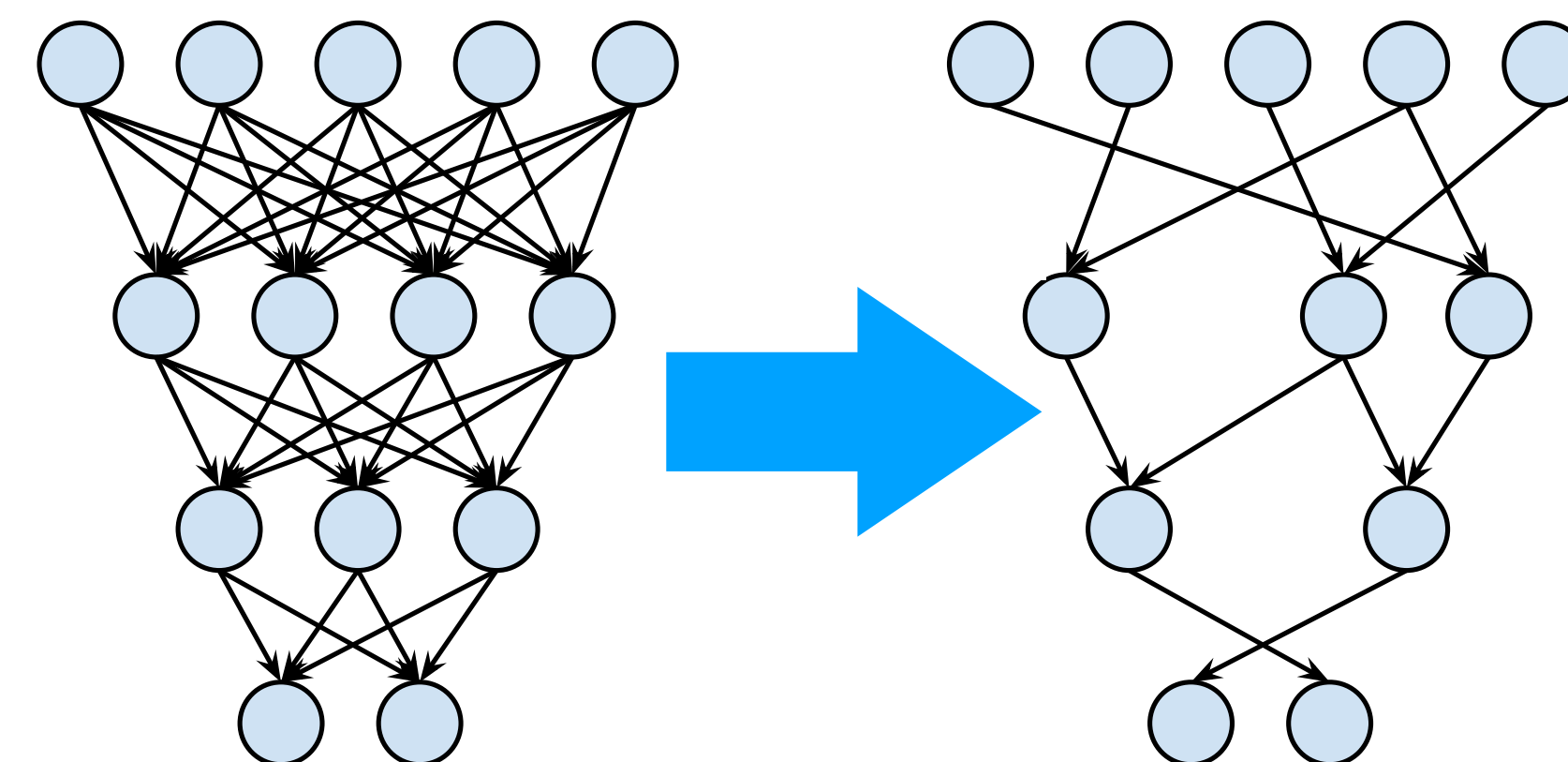
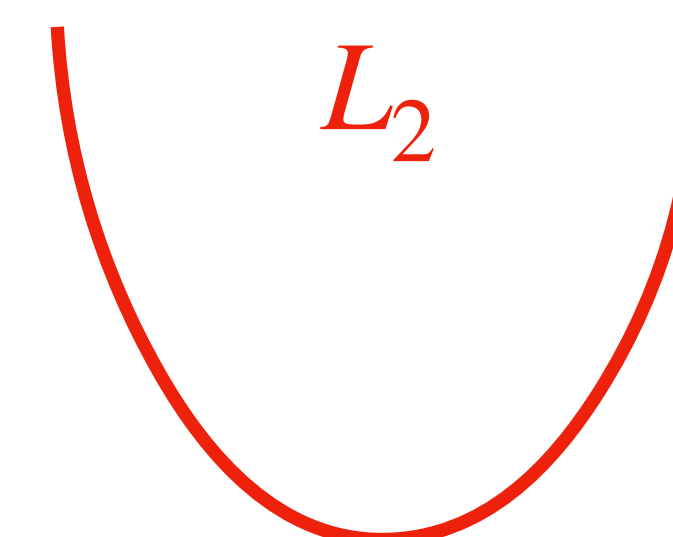
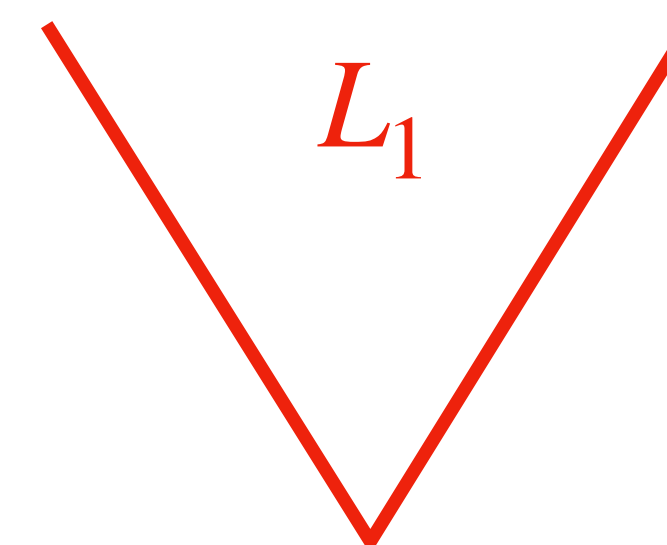
Underfitting vs Overfitting



Regularisation

- A *regularisation method* is a modification of the learning algorithm that is intended to reduce its generalization error but not its training error
- adding constraints to the parameters, e.g., *clipping* the range of values it can take
- adding an extra term in the loss, such as L_p *regularisation*
- modifying the network architecture, e.g., *dropout and pruning*
- Most of regularisation methods can be seen as priors on the parameters
- Sometimes they express a preference for a simpler (i.e., smaller) network architecture

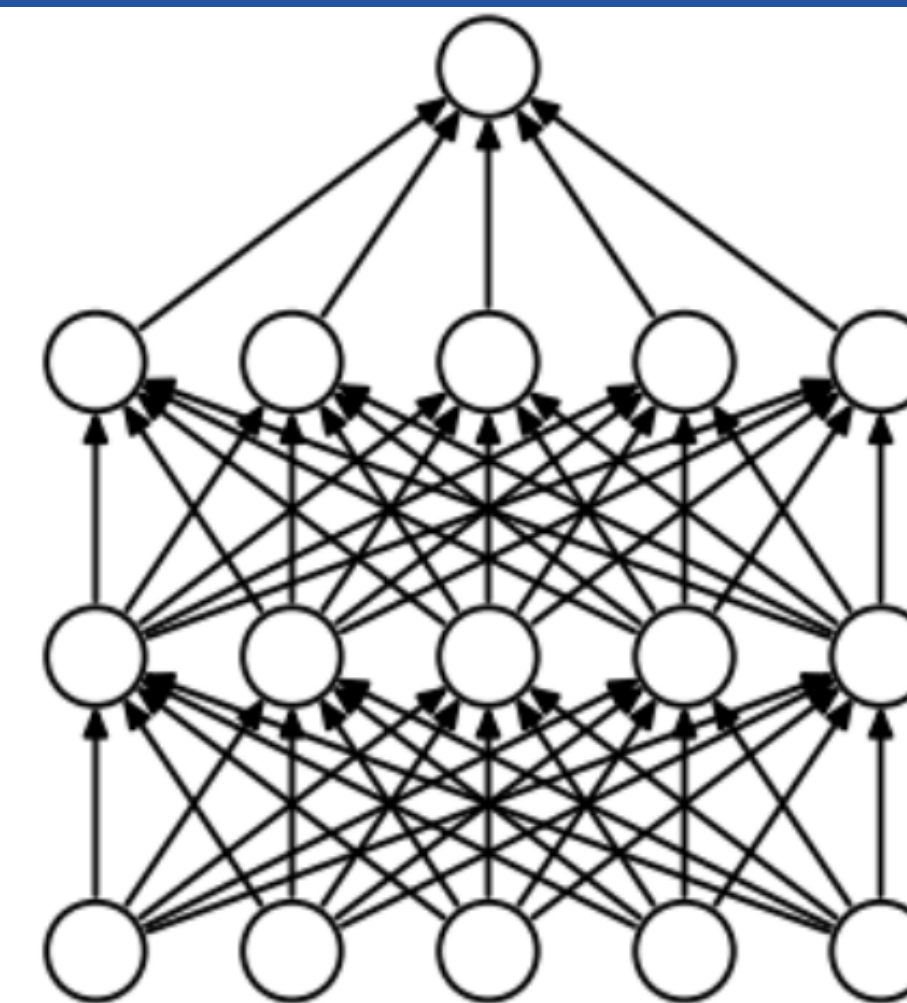
weight clipping



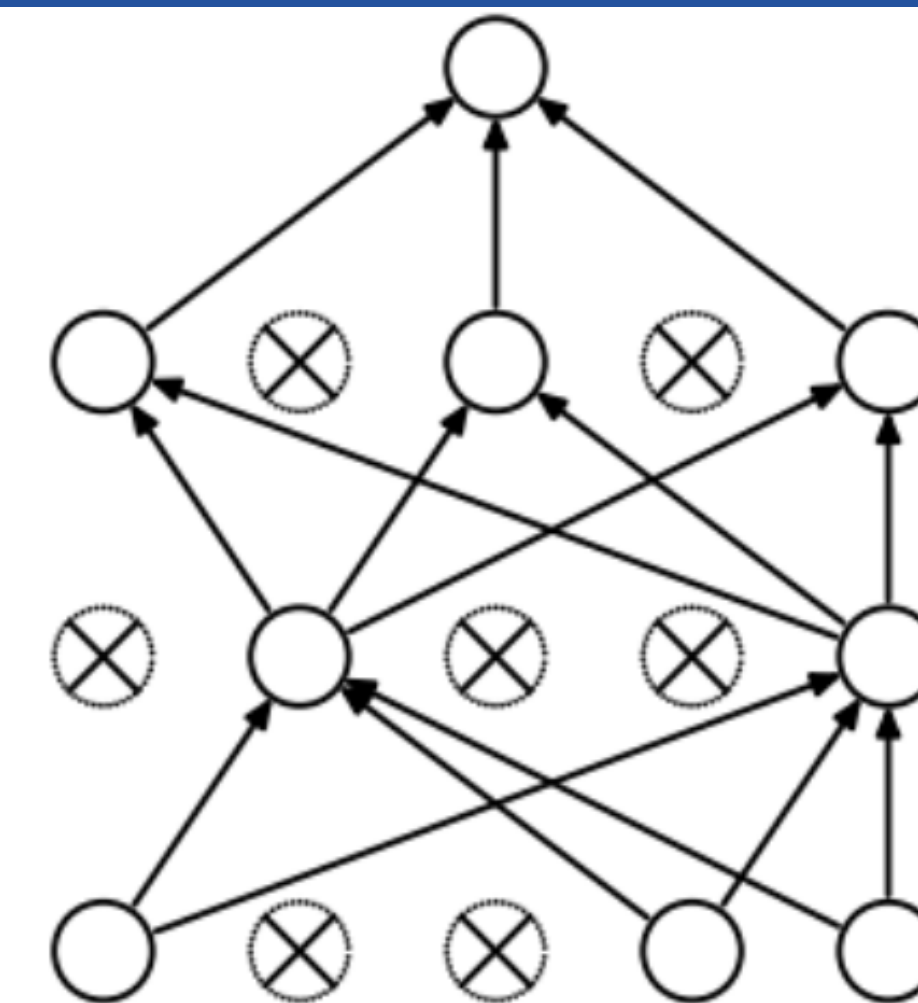
- ◎ *If the “right model” is among the models that your network can approximate, regularisation removes unnecessary degrees of freedoms*
 - ◎ *same as fitting a parabola with a 4th order polynomial*
- ◎ *But this is rarely the case*
 - ◎ *The “right” model is unaccessible, and most likely not among the functions that your network can approximate*
 - ◎ *In this case, one wants the largest/deepest (i.e., most complex) model possible*
 - ◎ *Even in these cases, regularisation is required to avoid overfitting*

Dropout

- Dropout is one of the most popular and easy-to-apply regularisation
- In Keras, it is just an extra layer you add
- It consists in randomly zero-ing some of the parameters during training
 - This is typically done by batch: when a new batch starts, all weights are back (to the values they had at the next-to-last batch) and new weights are randomly zero-ed
- The final network is obtained from the optimization of an ensemble of smaller networks that share parameters
- This prevents overtraining by brute force

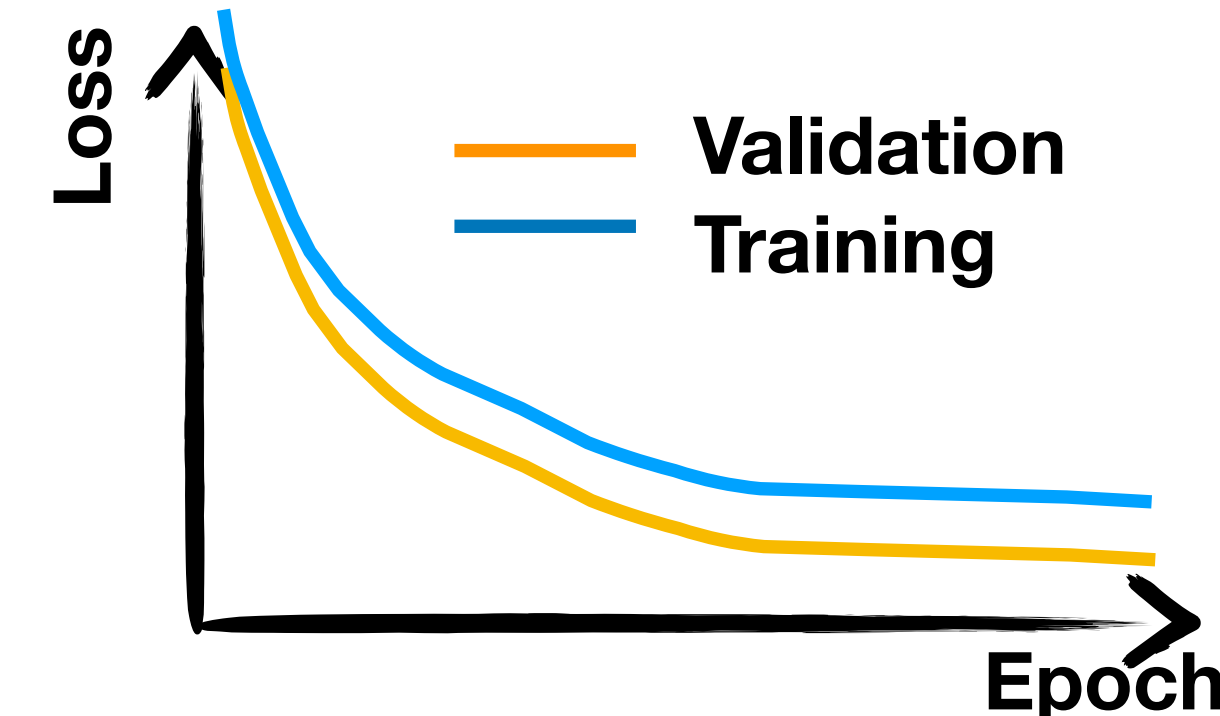
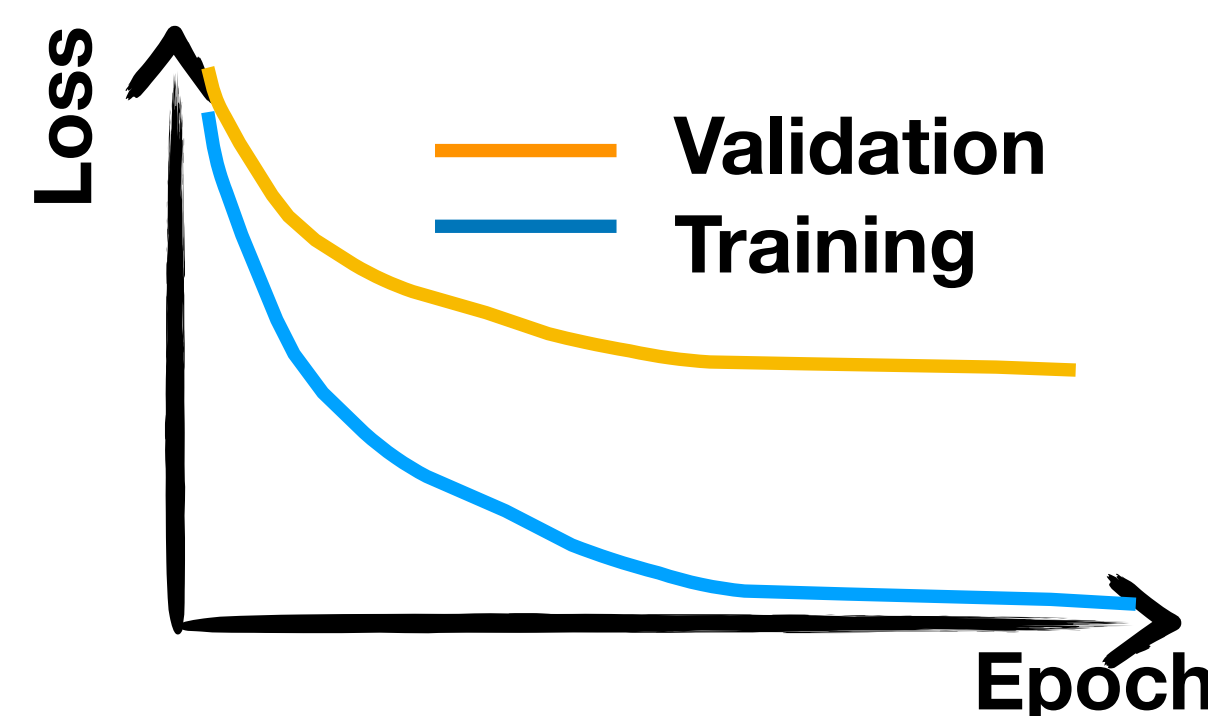


(a) Standard Neural Net



(b) After applying dropout.

Since the network is smaller at training time, training loss can become worse than validation loss



Parameter Norm penalty

- ⦿ This regularisation method consists in adding a term to the loss

$$\tilde{J}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) + \alpha \Omega(\boldsymbol{\theta})$$

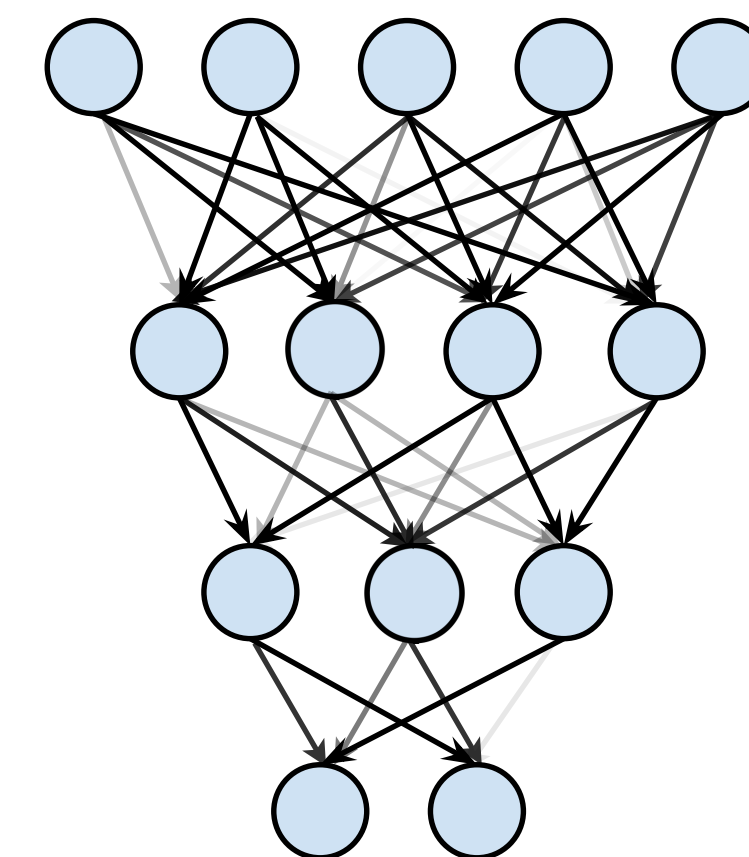
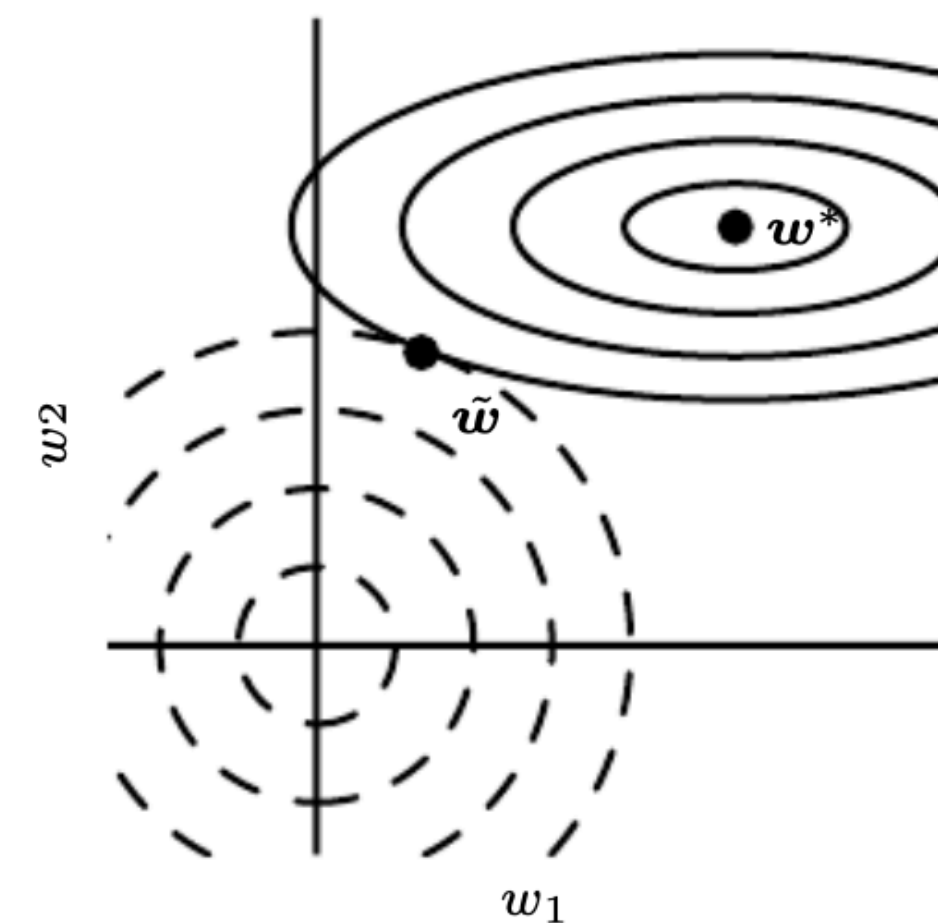
- ⦿ $\Omega(\theta)$ is some norm of the network parameters θ . For instance $L_2 = \sum_i |\theta_i|^2$

- ⦿ α is a hyperparameter that sets the relative importance of the regularisation

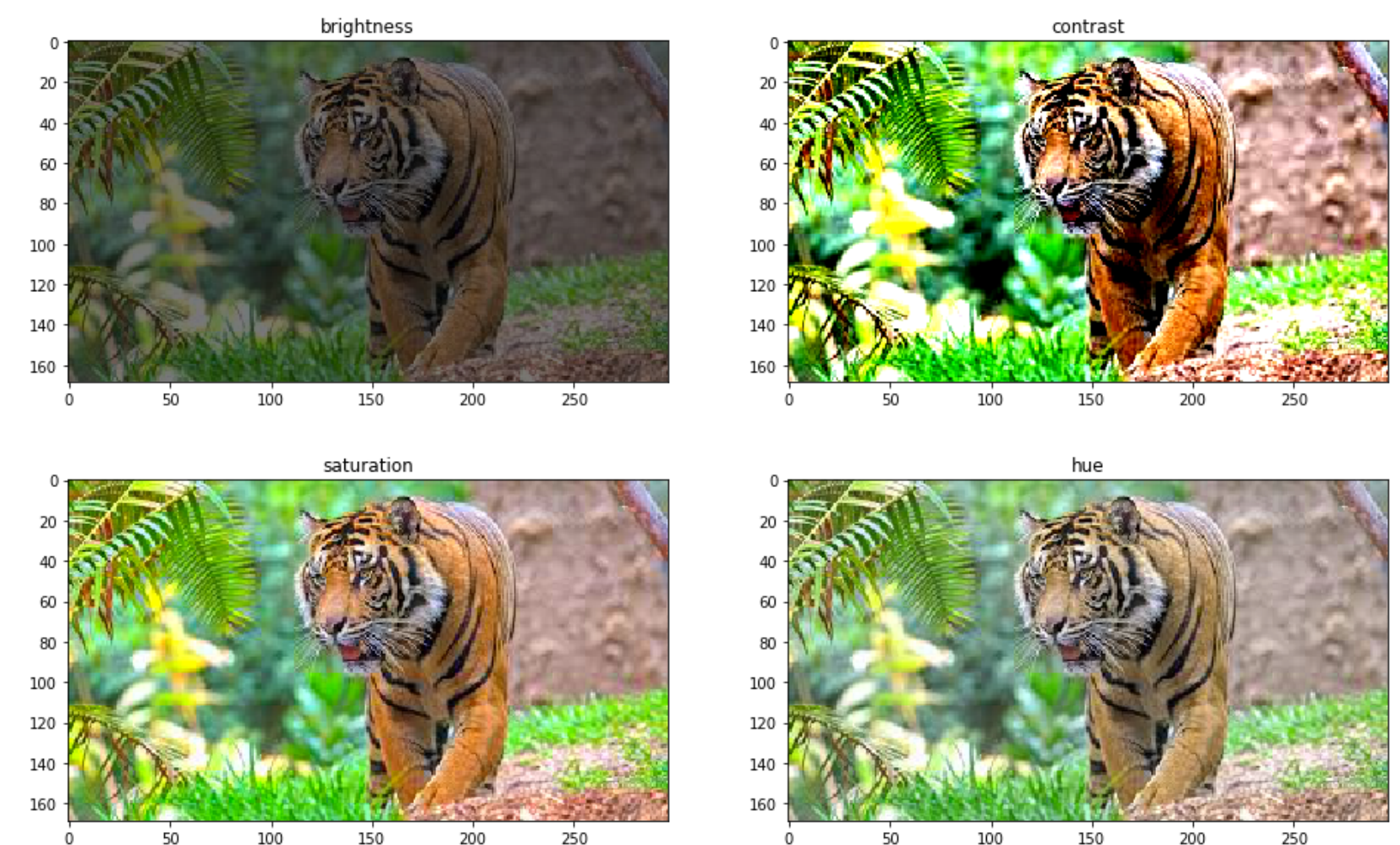
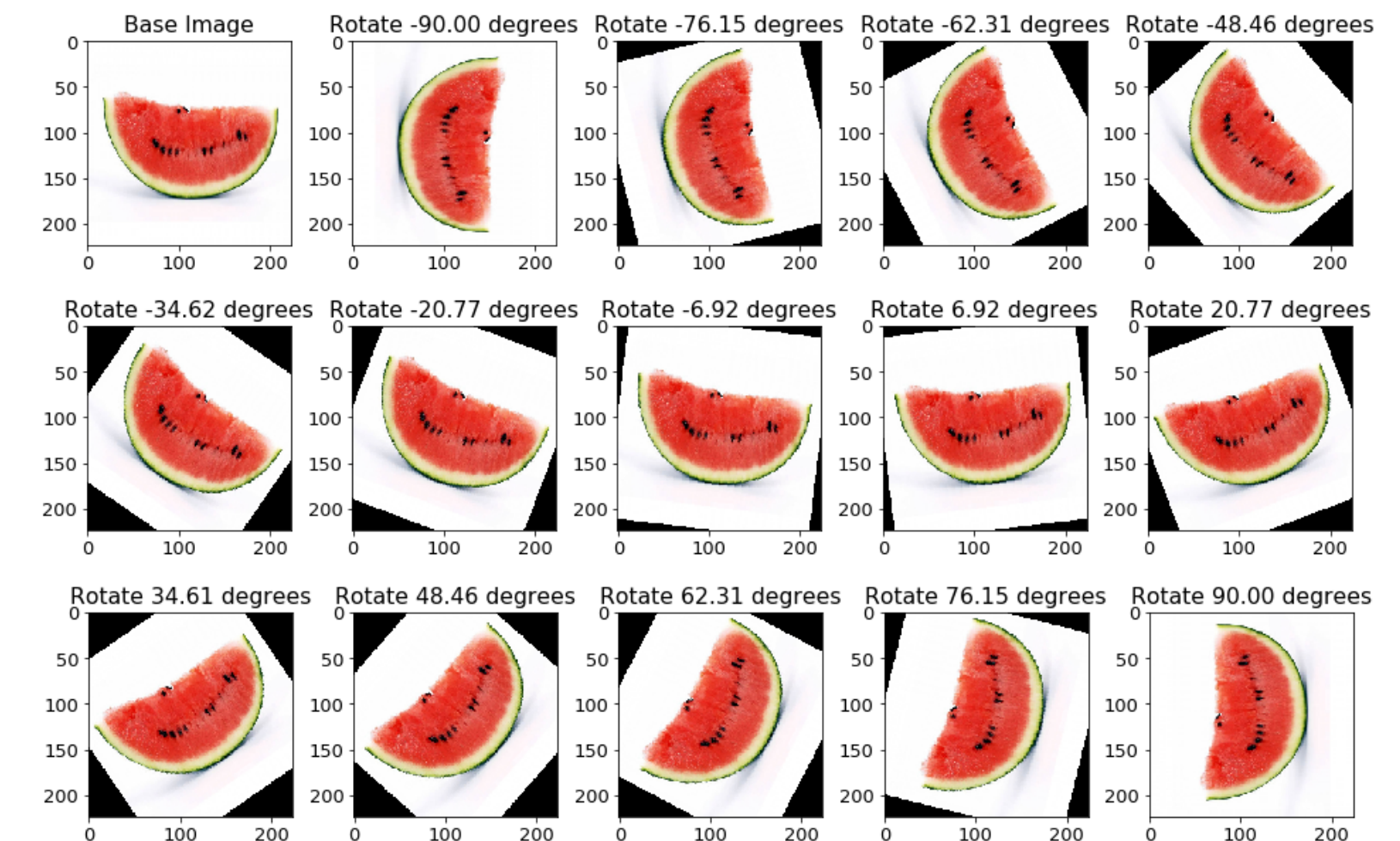
- ⦿ With deep networks, one often uses a different α for each layer

- ⦿ $\Omega(\theta)$ is a prior that privileges $\theta=0$, i.e., less degrees of freedom (easy to see with DNNs)

- ⦿ Similar to dropout in spirit, but replacing discrete yes/no switches with tunable gauges



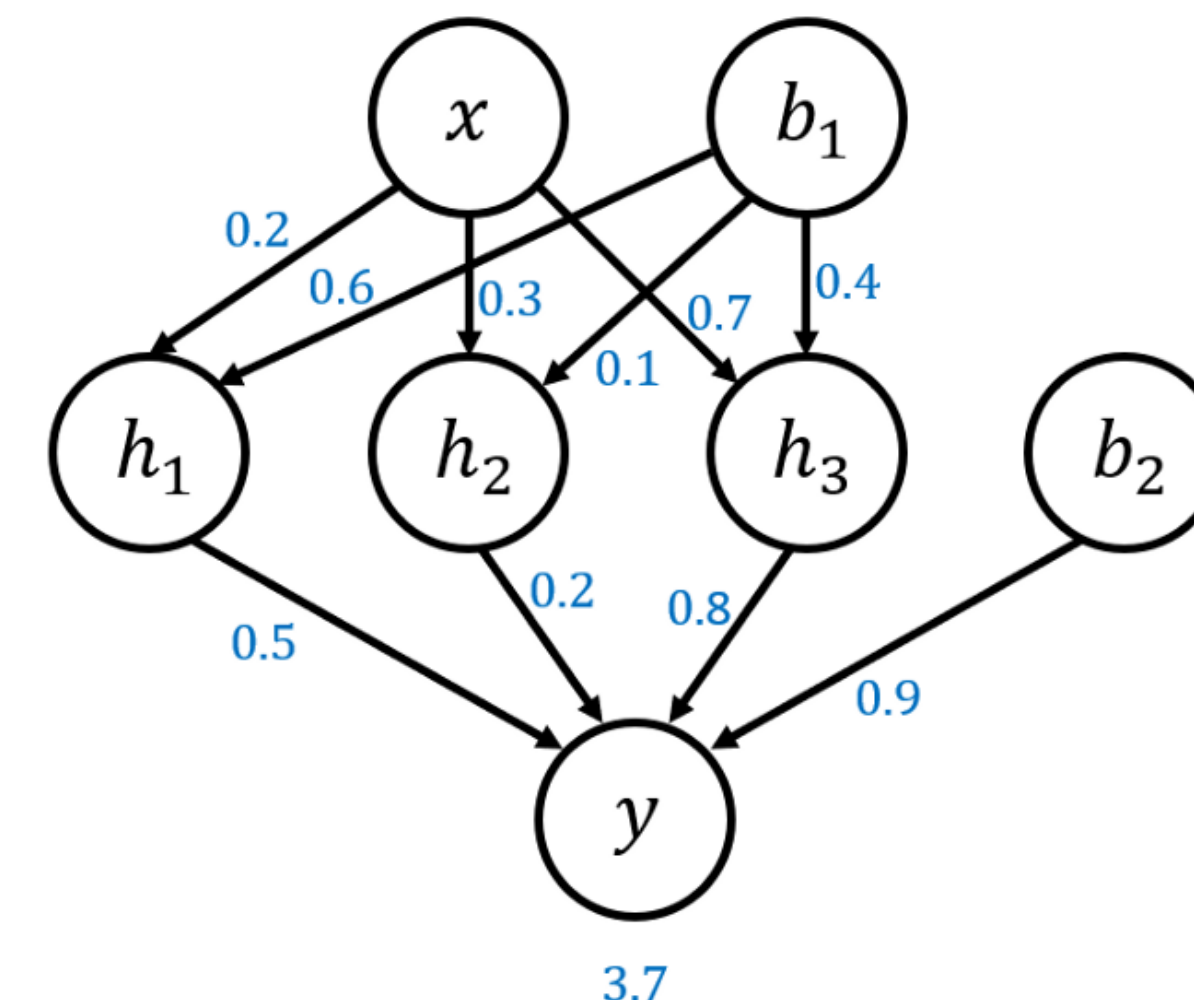
- Regularisation can also be applied on the dataset, rather than on the network or the loss function
- Data augmentation** consists in training on more data than those specific of the task
 - This approach is often used in classification problems
- Example: with images, one can augment the data applying **data transformations** (translations, rotations, etc.) to the original images.
 - It helps even when the architecture is equivariant to that transformation
 - But it could generate confusion: e.g, a 180° rotate 6 looks like a 9, which is a problem for a MNIST classifier
- Sometimes the transformation can be applied using more complex **filters**



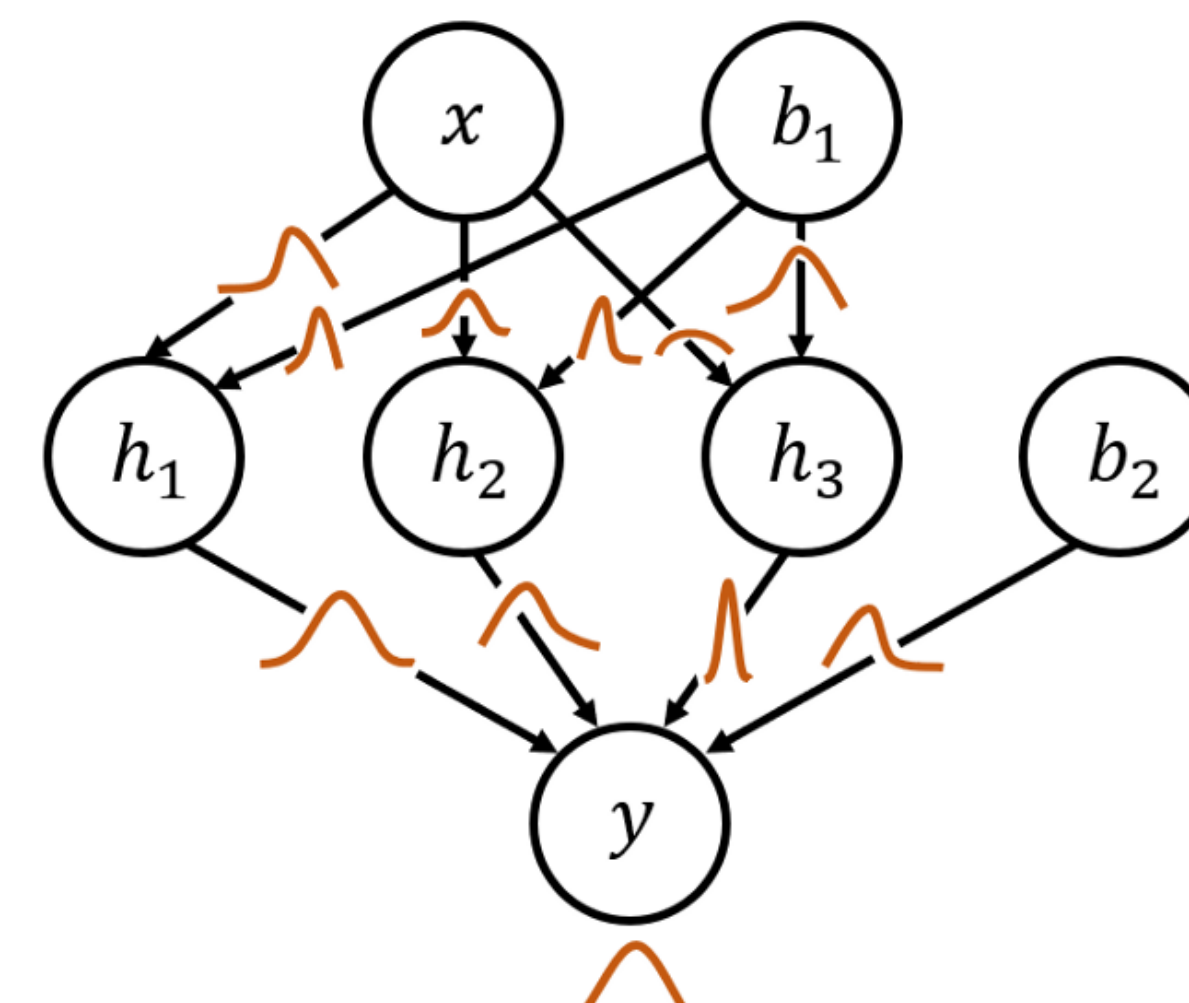
Noise on weights

- One way to regularise a network and gain generalization power is by perturbing the weight
- Replace a fixed value θ with $\hat{\theta} = \theta + \epsilon$, ϵ being some random number
 - Could be distributed flat in a range
 - Could be distributed as a Gaussian, e.g., $\mathcal{N}(0,1)$
- This procedure replaces a neural network with an ensemble of networks, obtained perturbing the weight set θ around it's central values
- These networks are called **Bayesian Neural Networks**

Standard Neural Network



Bayesian Neural Network



- ◎ *Often, data labelling (e.g., classification done by humans) comes with mistakes*
- ◎ *A network would take these mistakes as ground truth*
- ◎ *To prevent mistakes from misguiding the learning process, one can try to model the mistake probability and add it to the learning procedure*
- ◎ *Example: trade the 0,1 “hard labels” of a binary classifier with “smooth labels” $\epsilon, 1 - \epsilon$, with $\epsilon \propto \mathcal{N}(0, \sigma_\epsilon)$ and $\sigma_\epsilon < 1$ (Gaussian centred at 0 and with RMS smaller than one)*

Noise on the data

- One way to regularise a network and gain generalization power is by perturbing the data with noise
 - One can try to emulate various weather conditions (e.g., light exposure in automotive)
 - Or emulate various sound quality for speech recognition
- This is usually done applying **random noise** to the inputs (random numbers distributed according to some prior, e.g., a Gaussian $\mathcal{N}(0,1)$)
- Sometimes one applies **structured noise**, e.g., noise designed to maximally confuse a classifier
 - This kind of noise is derived with so-called **adversarial attacks** (we will discuss them in ~ 1 month)

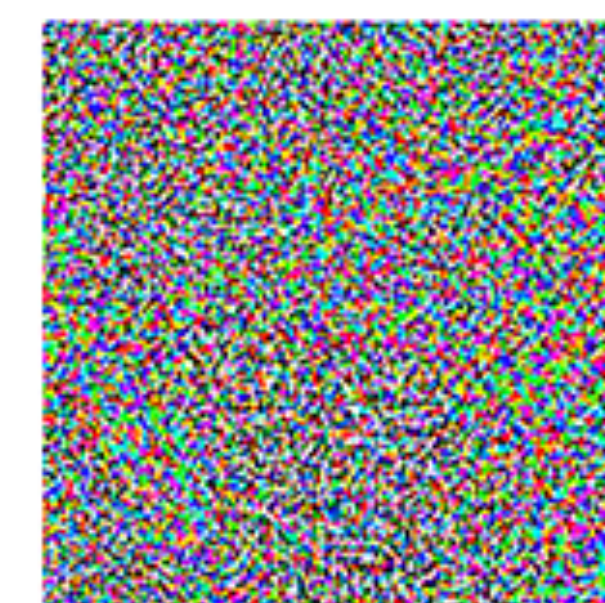


x

“panda”

57.7% confidence

+ .007 ×

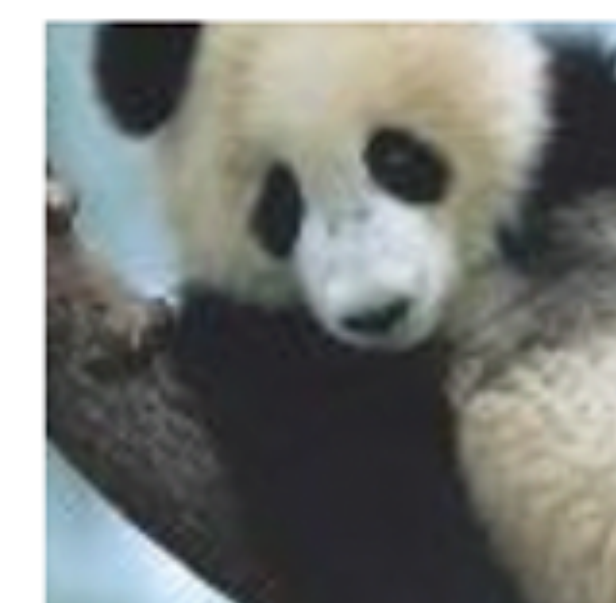


$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

=



$x +$

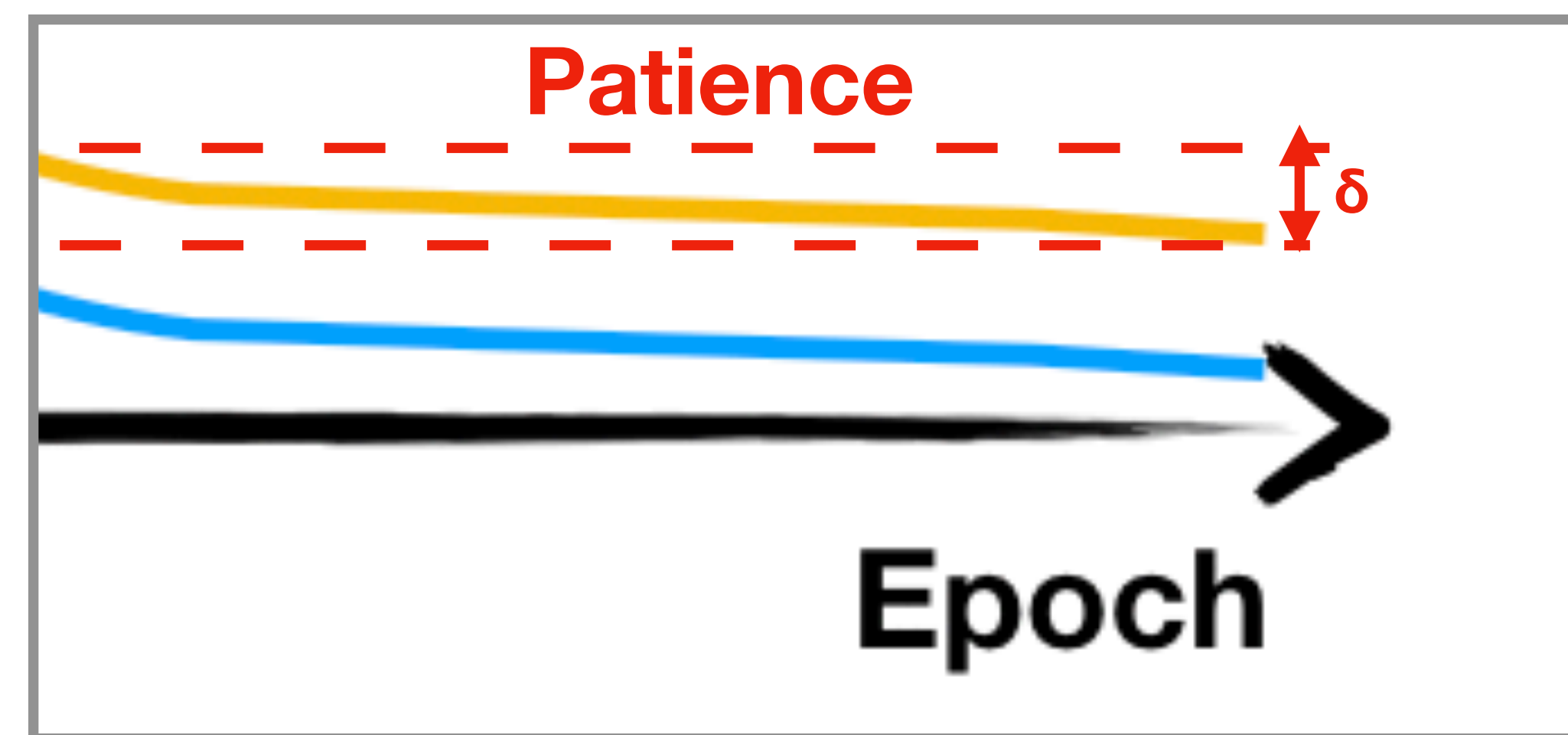
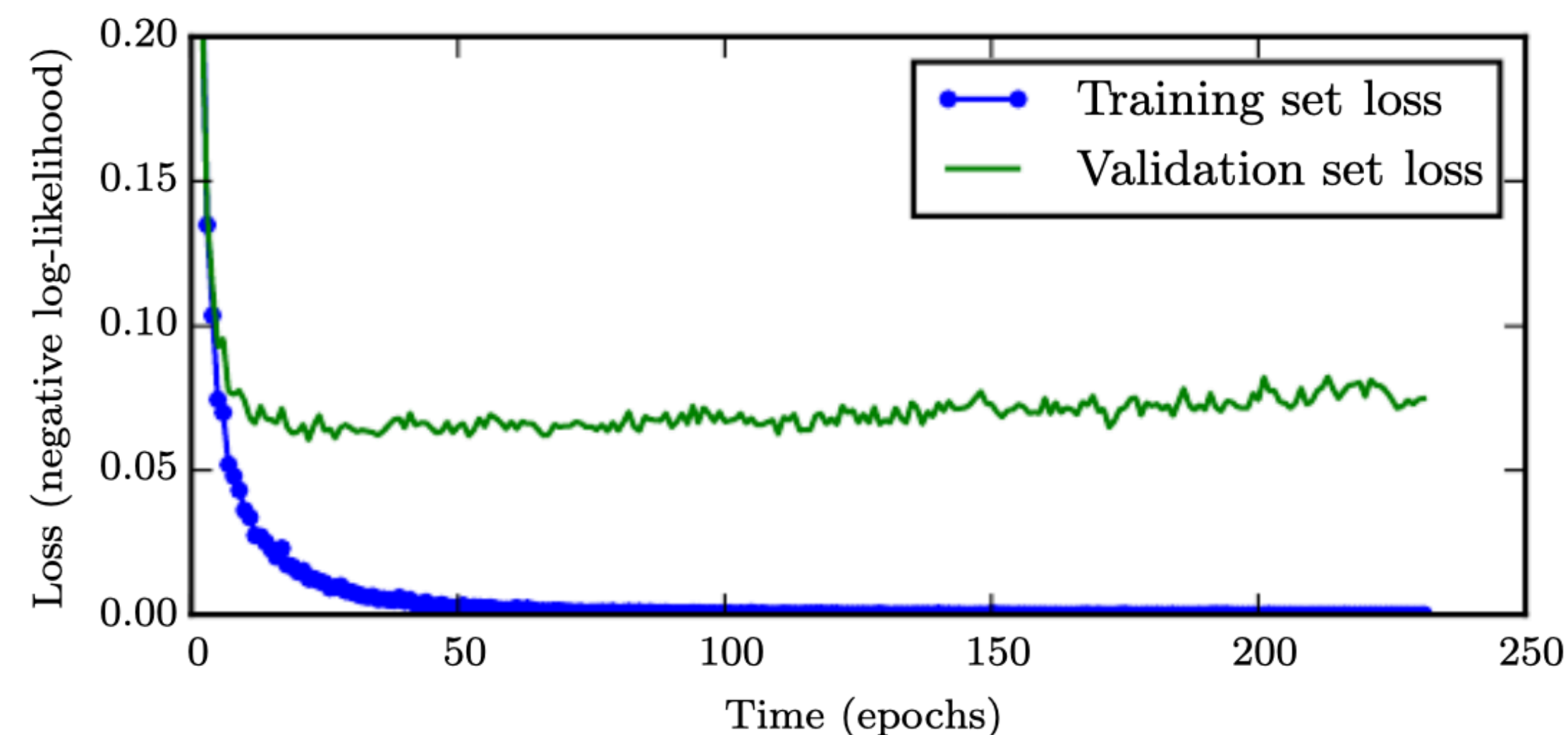
$\epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

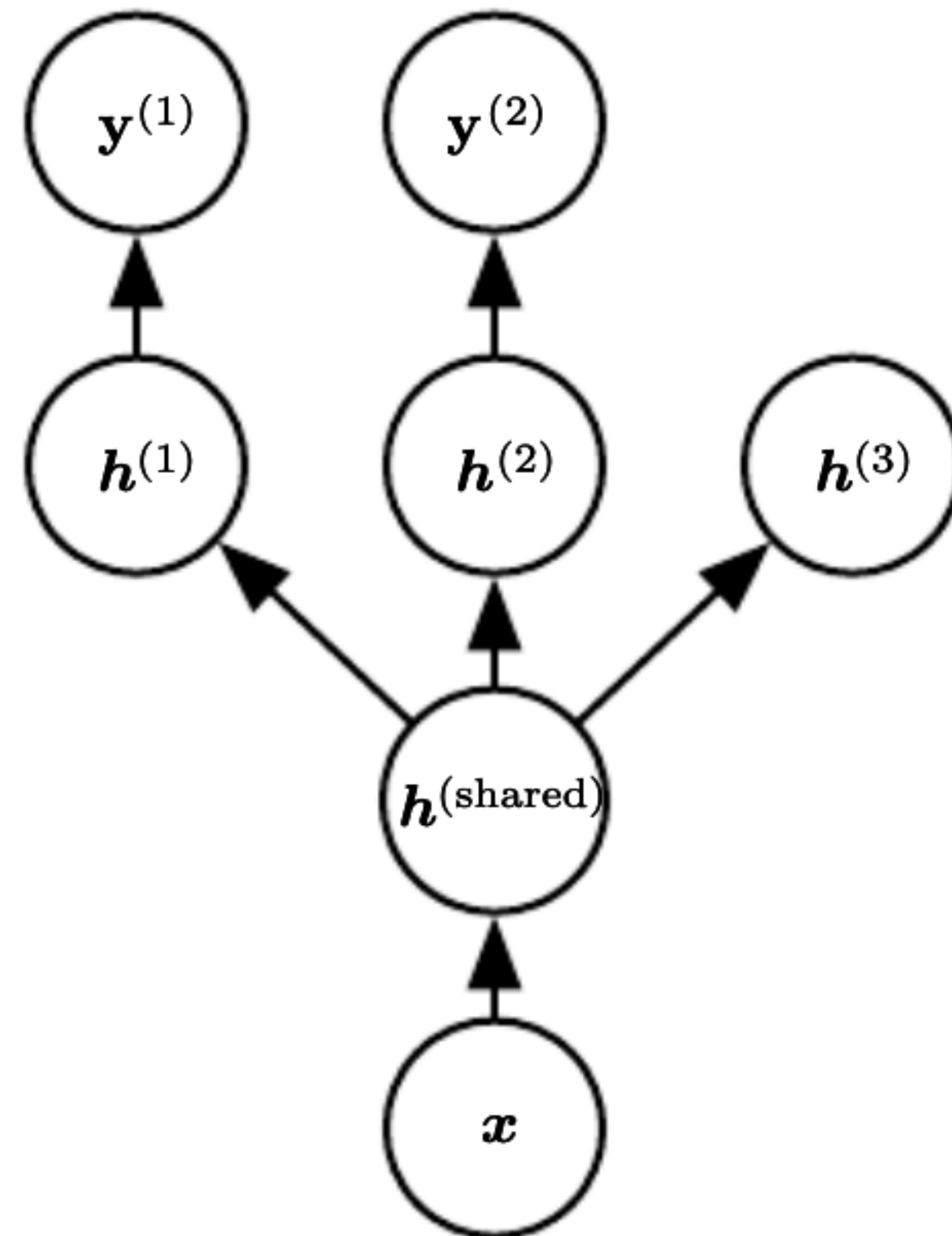
99.3 % confidence

Early Stopping

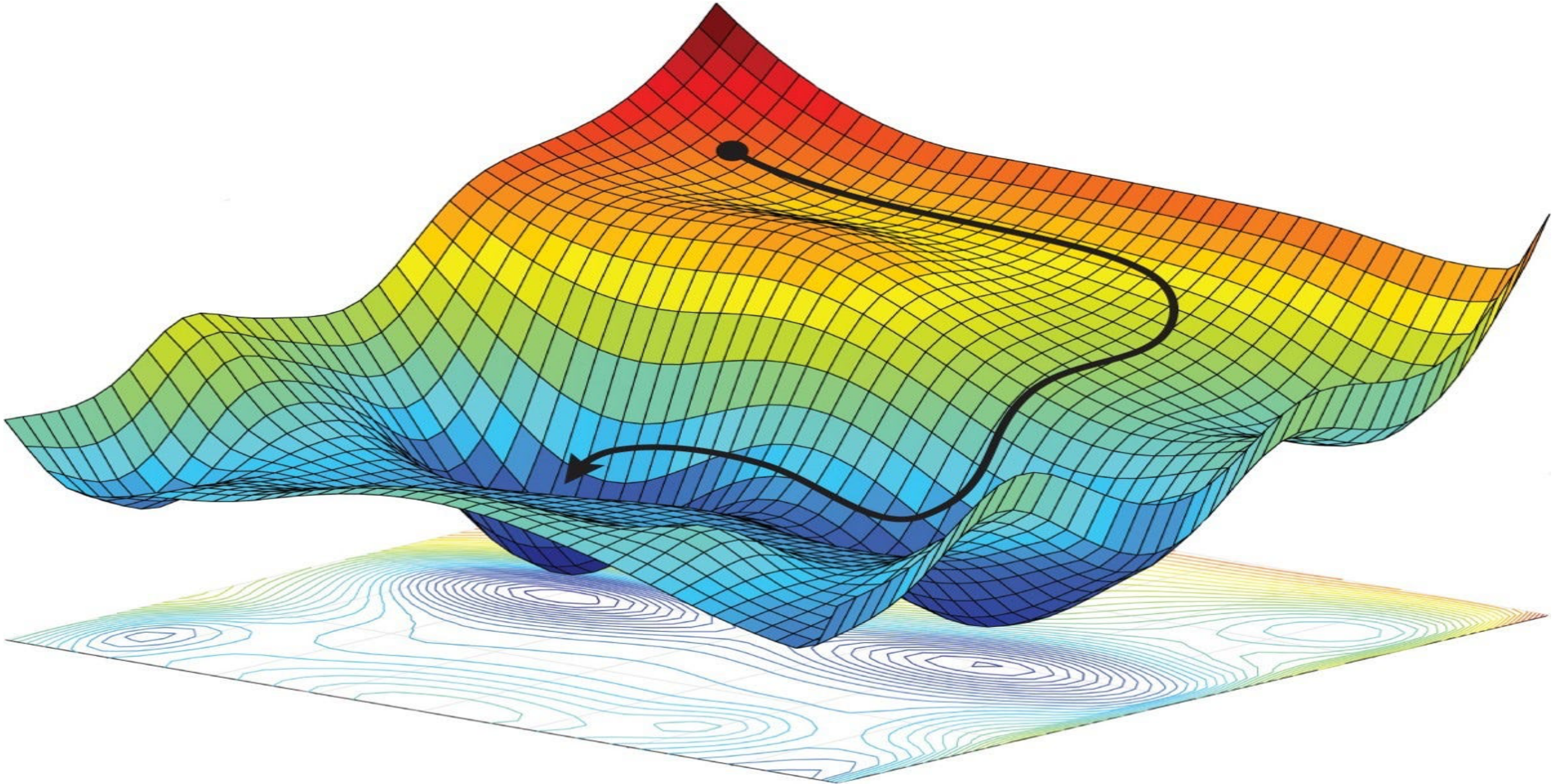
- ⦿ Sometimes the val loss grows after a little bit, while the training loss might still decrease
- ⦿ Having stopped the training earlier, we would have had a better generalization loss
- ⦿ This can be done adding early stopping to the training: *stop the training if after p epochs (the patience) the validation loss did not increase by more than a fixed amount δ*
- ⦿ The number of epochs is yet another hyperparameter and early stopping is an algorithm that scans and fixes this hyperparameter
- ⦿ Since the outcome is a better generalization power (smaller distance between training and validation), early stopping is also a regularisation algorithm



- ◉ Sometimes, making the task come complex prevents overtraining
- ◉ Example: you want to classify particles
 - ◉ Your discrimination power changes with the energy of the particle
 - ◉ By learning also the particle energy, you learn this dependence and make the classification more robust
- ◉ This is typically realised with two DNNs sharing the same features (e.g., extracted from the same convolutional layers)
- ◉ It only makes sense when the two tasks are related: among the factors that explain the variations observed in the data associated with the different tasks, some are shared across two or more tasks



Optimization



The Empirical risk

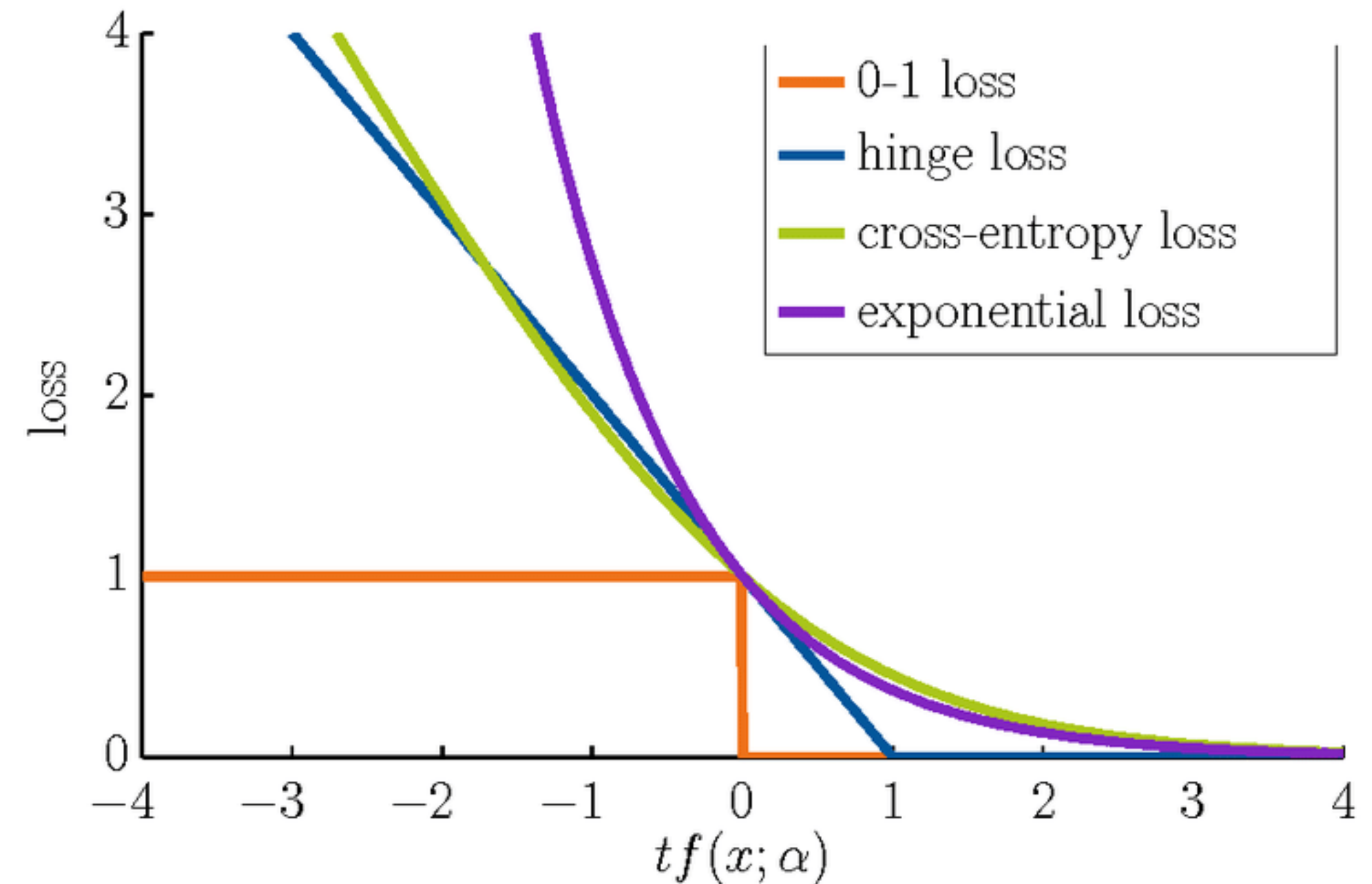
- ◎ *The goal of the optimization step is the minimisation of the expected generalization error. This quantity is known as the **risk***
- ◎ *You want to minimise the probability to make mistakes on an independent dataset (e.g., your test sample)*
- ◎ *We can only do it exactly if we know the true data distribution (which we never do)*
- ◎ *Instead, we estimate it numerically, measuring the risk on a training dataset. This is called the empirical risk*

$$\mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}(\mathbf{x}, y)} [L(f(\mathbf{x}; \boldsymbol{\theta}), y)] = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}),$$

Remember: Empirical risk = cost = loss = ...

Surrogate loss functions

- ⊙ Sometimes problems can be *intractable*
- ⊙ e.g., if the loss is not a smooth convex function it can't be easily minimized.
- ⊙ Example: % of correct 0-1 classification (accuracy, aka 0-1 loss) is discontinuous
- ⊙ Instead, the negative log likelihood is typically used, resulting in the binary cross entropy



Example: Cross Entropy

- *Bernoulli's problem: probability of a process that can give 1 or 0*

$$\mathcal{L} = \prod_i p_i^{x_i} (1 - p_i)^{1-x_i}$$

- *The corresponding likelihood is (as usual) the product of the probabilities across the events*

$$-\log \mathcal{L} = -\log \left[\prod_i p_i^{x_i} (1 - p_i)^{1-x_i} \right]$$

- *Maximizing the likelihood corresponds to minimizing the $-\log \mathcal{L}$*

- *Minimizing the $-\log \mathcal{L}$ corresponds to minimizing the binary cross entropy*

$$= - \sum_i \left[x_i \log p_i + (1 - x_i) \log (1 - p_i) \right]$$

- *We will use this expression as loss function in classification problems*

REMINDER FROM LECTURE 3

Batches and Minibatches

- Loss is normally decomposed in a sum over the training set

- Remember: the advantage of $-\log\text{Lik}$:

$$\arg \max \left[\log \prod_i p_i \right] = \arg \min \left[- \sum \log p_i \right] = \arg \min \mathbb{E} [\log p]$$

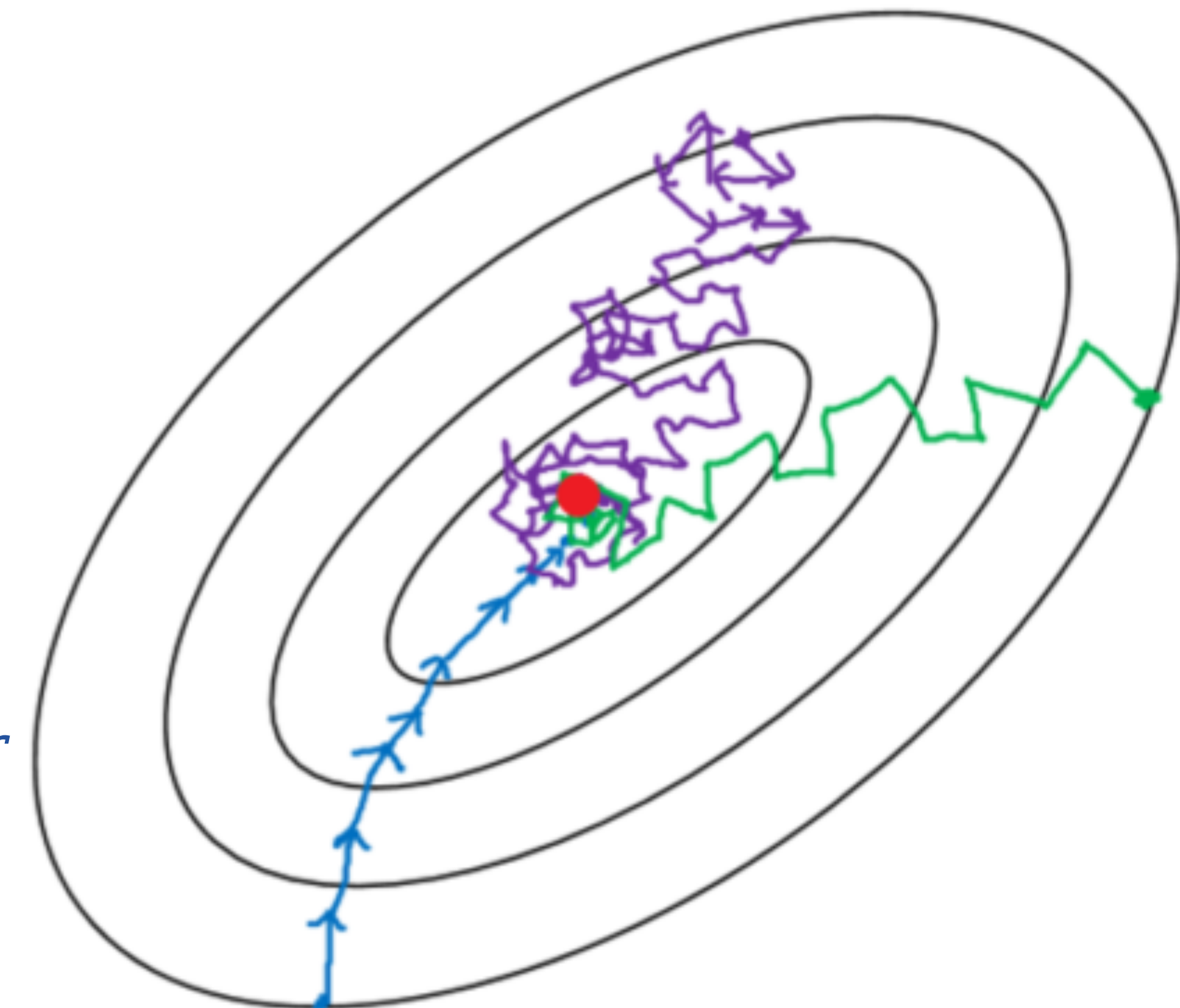
- Computationally very expensive to perform the **average on the whole training set (batch minimisation)**

- But we know that the RMS of an average is $\sigma/\sqrt{n}$, when n is the number of examples and σ is the RMS of the sample

- Estimates on x10 more data are x10 more expensive but only x3.16 more precise

- one can then perform single-example estimates and update the model (stochastic models), with large parallelizability at large stochasticity cost

- One can instead perform the average on small portions of data (minibatch) in parallel, gaining in performance while reducing stochastic noise



- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent

Algorithm 8.1 Stochastic gradient descent (SGD) update

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

$k \leftarrow k + 1$

end while

Learning rate



A few considerations

- ⦿ *Larger batches provide a more accurate estimate of the gradient, but with less than linear returns*
- ⦿ *Multicore architectures are usually underutilized by extremely small batches. This motivates using some absolute minimum batch size, below which there is no reduction in the time to process a minibatch*
- ⦿ *If all examples in the batch are to be processed in parallel (as is typically the case), then the amount of memory scales with the batch size. For many hardware setups this is the limiting factor in batch size*
- ⦿ *Some kinds of hardware achieve better runtime with specific sizes of arrays. Especially when using GPUs, it is common for power of 2 batch sizes to offer better runtime. Typical power of 2 batch sizes range from 32 to 256, with 16 sometimes being attempted for large models*
- ⦿ *Small batches can offer a regularizing effect, due to the noise they add to the learning process. Generalization error is often best for a batch size of 1. Training with such a small batch size might require a small learning rate to maintain stability because of the high variance in the estimate of the gradient. The total runtime can be very high as a result of the need to make more steps, both because of the reduced learning rate and because it takes more steps to observe the entire training set*
- ⦿ *NNs learning process is not an understood process. It is then difficult to predict which choice will result in a better convergence*

- ◎ *The result of the learning process may depend on where you start from*
- ◎ *Examples: all parameters are set to same values and two units are interchangeable (same connections, same activation, etc). They might introduce some degeneracy in the training. Better choose **DIFFERENT INITIALISATIONS to break the symmetry***
- ◎ *usually one takes random values from Gaussian or flat distribution*
- ◎ *large values break degeneracy more*
- ◎ *but too large values can cause numerical instabilities*
- ◎ *The scale of the random generation is often used as a hyperparameter*

Updating the learning rate

⊙ Usually, the learning rate is adapted to the training history (smaller and smaller to increase accuracy once getting close to minimum)

⊙ Various prescriptions:

⊙ AdaGrad: use as learning rate $\frac{\epsilon}{\delta + \sqrt{r}}$, where ϵ is the global learning rate (as in SGD), δ is a numerical constant for stability (to avoid /0) and r is the cumulative gradient

$$\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}, \quad \Delta \boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$$

⊙ RMSPROP: as AdaGrad, but with hyperparameter ruling relative importance between cumulative gradient and last term of sum

$$\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$$

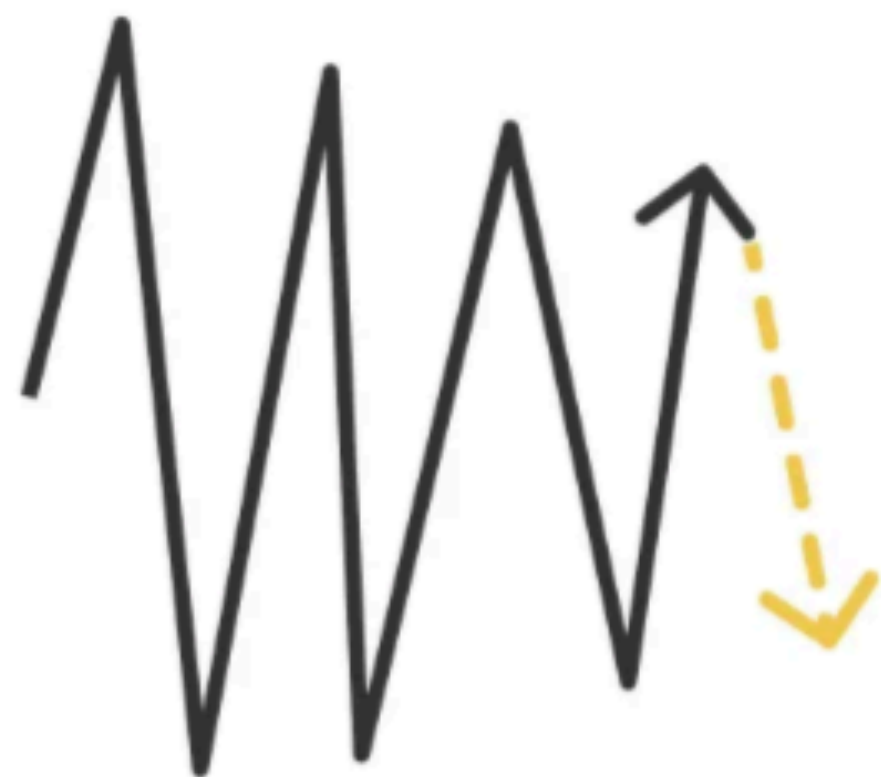
⊙ Adam: Cumulative gradient as in RMSPROP, but also uses momentum for gradient update (to avoid getting stuck)

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \frac{\epsilon}{\sqrt{\mathbf{r}}} \odot \mathbf{g}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$$

⊙ How to choose the best algorithm? Hyperparameter choice (see [here](#))

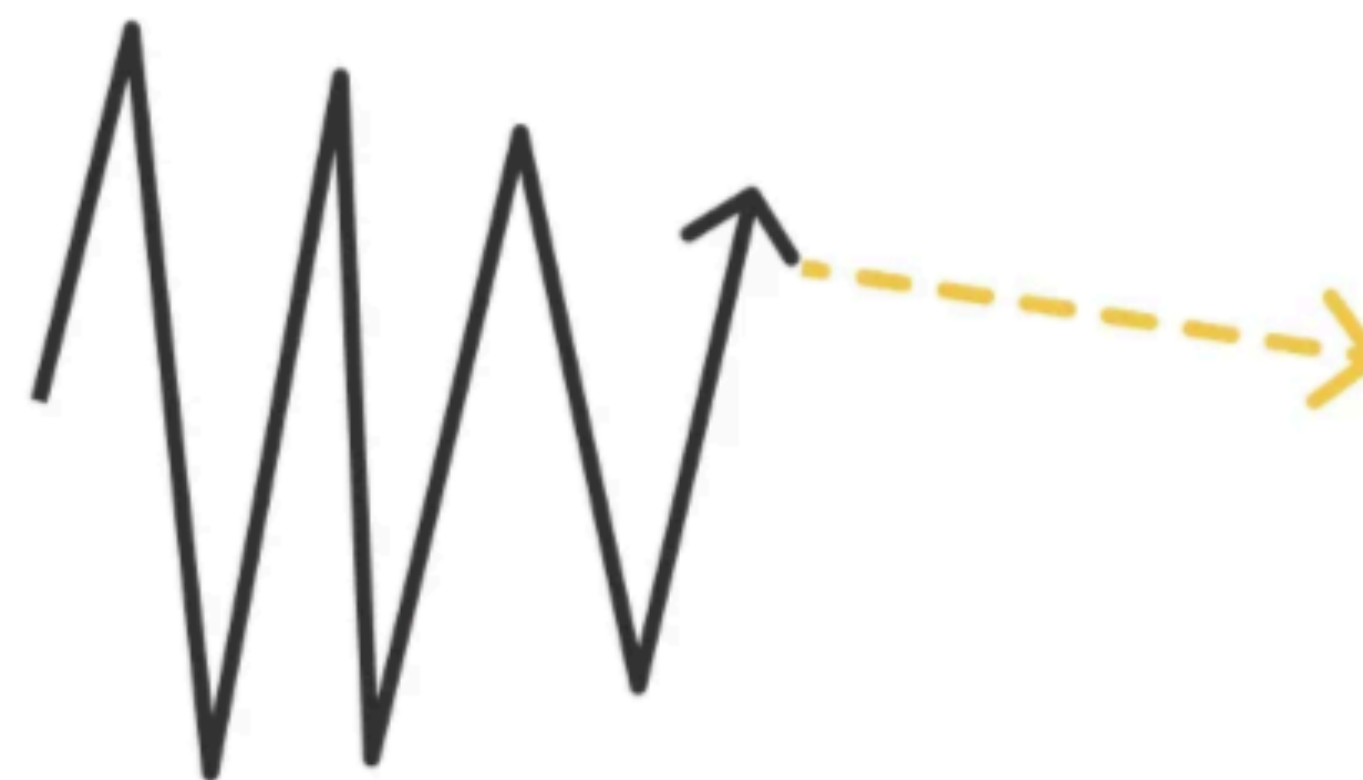
Updating the learning rate

Gradient descent



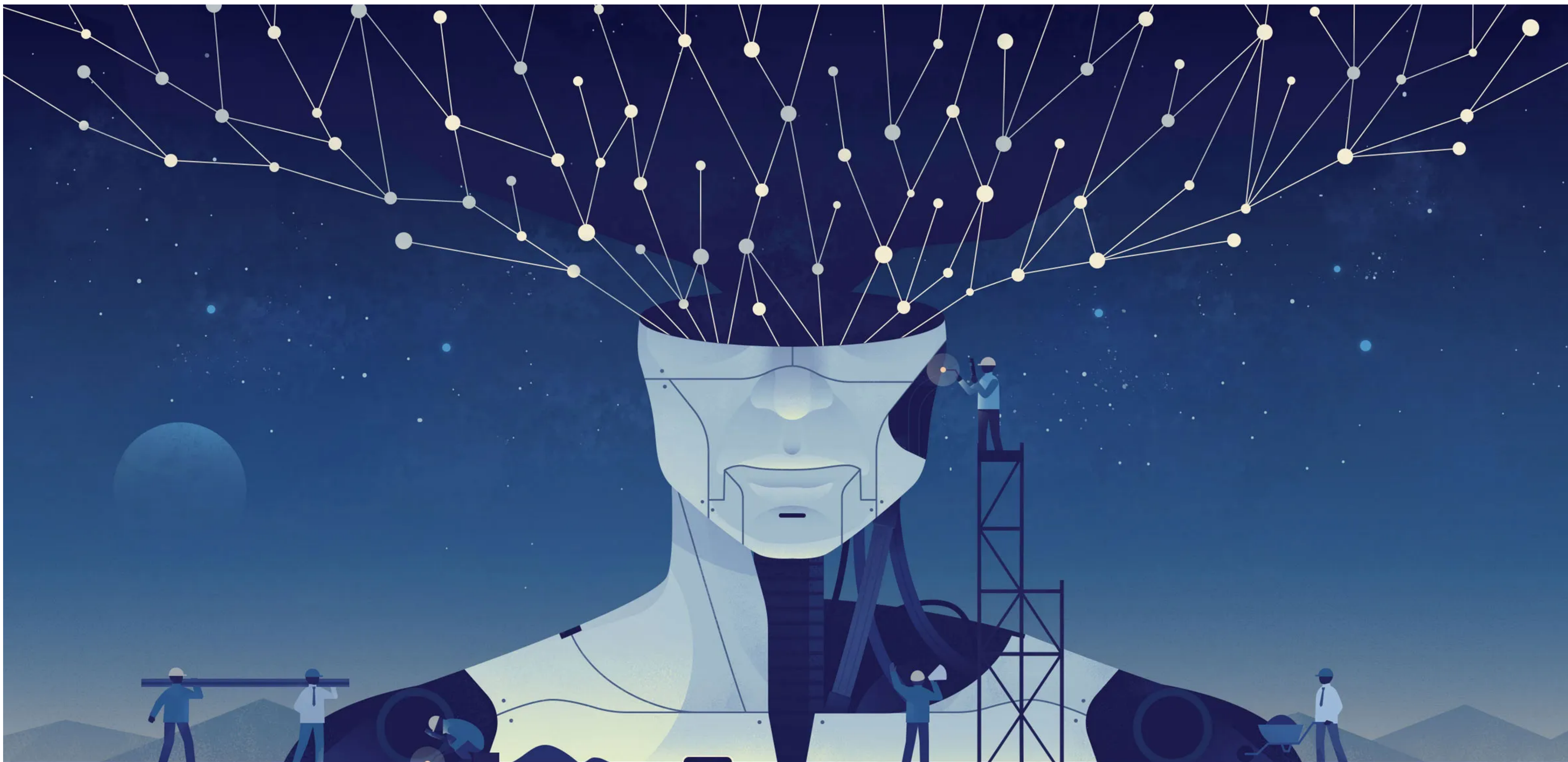
the gradient computed on the current iteration does not prevent gradient descent from oscillating in the vertical direction

Momentum



the average vector of the horizontal component is aligned towards the minimum

the average vector of the vertical component is close to 0



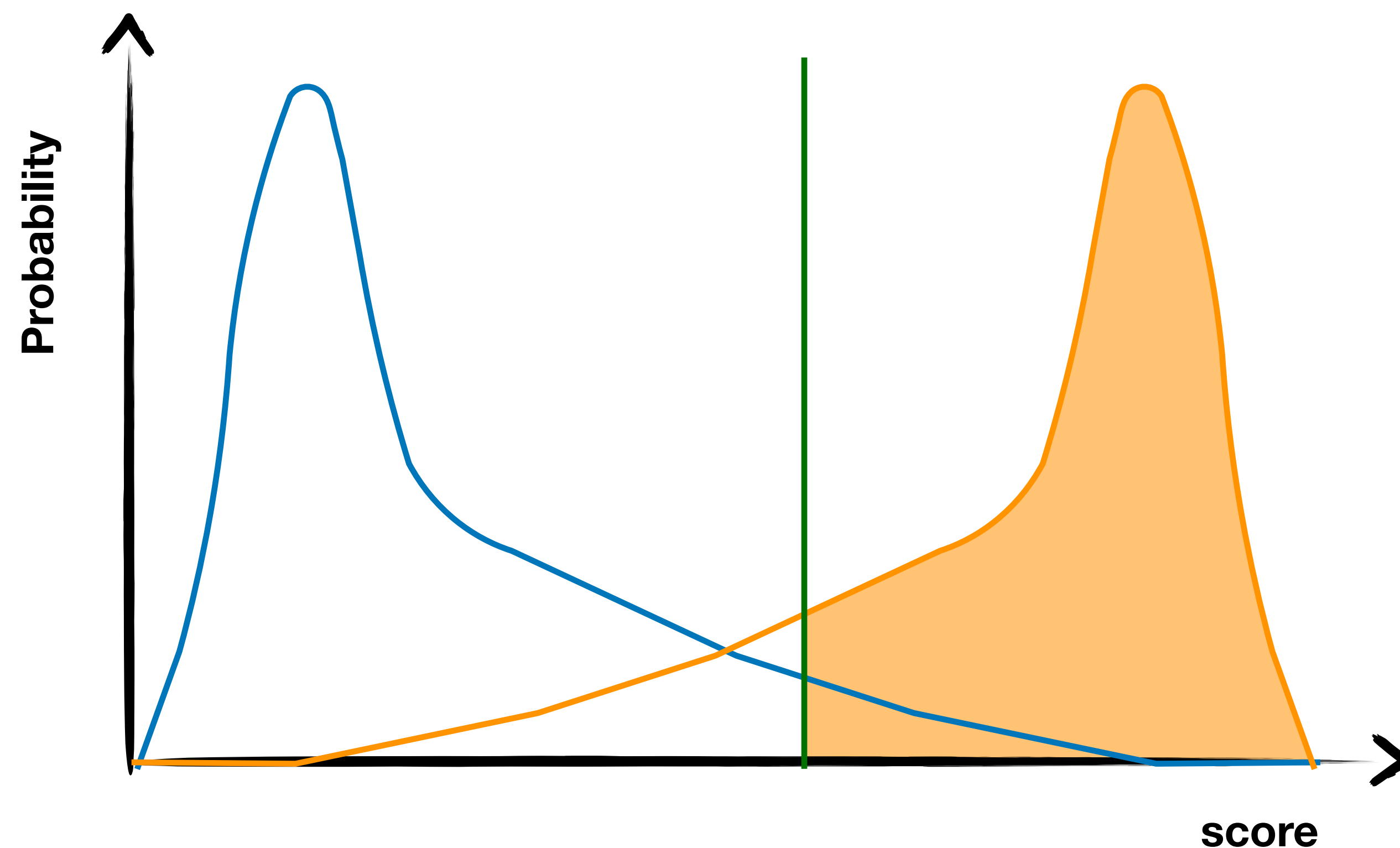


What to do in practice

- ◎ *This is not an exact science:*
 - ◎ *You can know all ingredients but need expertise to put them together*
 - ◎ *Often, you need to try things*
- ◎ *Get a working pipeline as soon as possible and try various options*
- ◎ *Record performance looking at various figures of metric*
 - ◎ *Don't get obsessed with one (accuracy, AOC, etc)*
- ◎ *Keep in mind numerical aspects of your minimization*
 - ◎ *Avoid very large/big numbers*
 - ◎ *stay away from zero gradients*
 - ◎ *If you can't learn, check for NaN*

Classifier metrics

- ⦿ A given threefold defines the following qualities
 - ⦿ *True-positives: Class-1 events above the threshold*
 - ⦿ *True-negatives: Class-0 events below the threshold*
 - ⦿ *False-positives: Class-0 events above the threshold*
 - ⦿ *False-negatives: Class-1 events below the threshold*



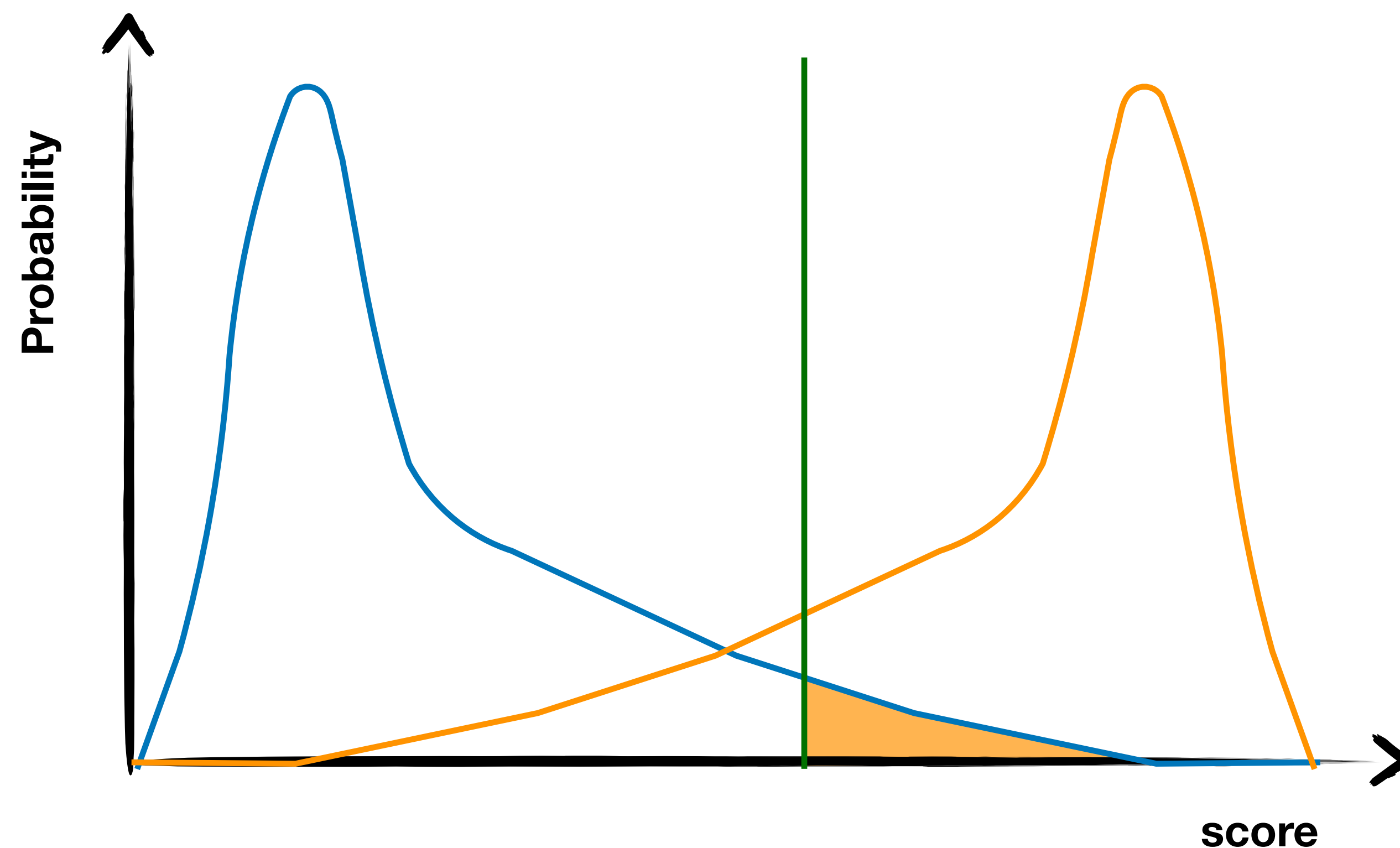
Classifier metrics

- A given threefold defines the following qualities
 - True-positives: Class-1 events above the threshold
 - True-negatives: Class-0 events below the threshold
 - False-positives: Class-0 events above the threshold
 - False-negatives: Class-1 events below the threshold



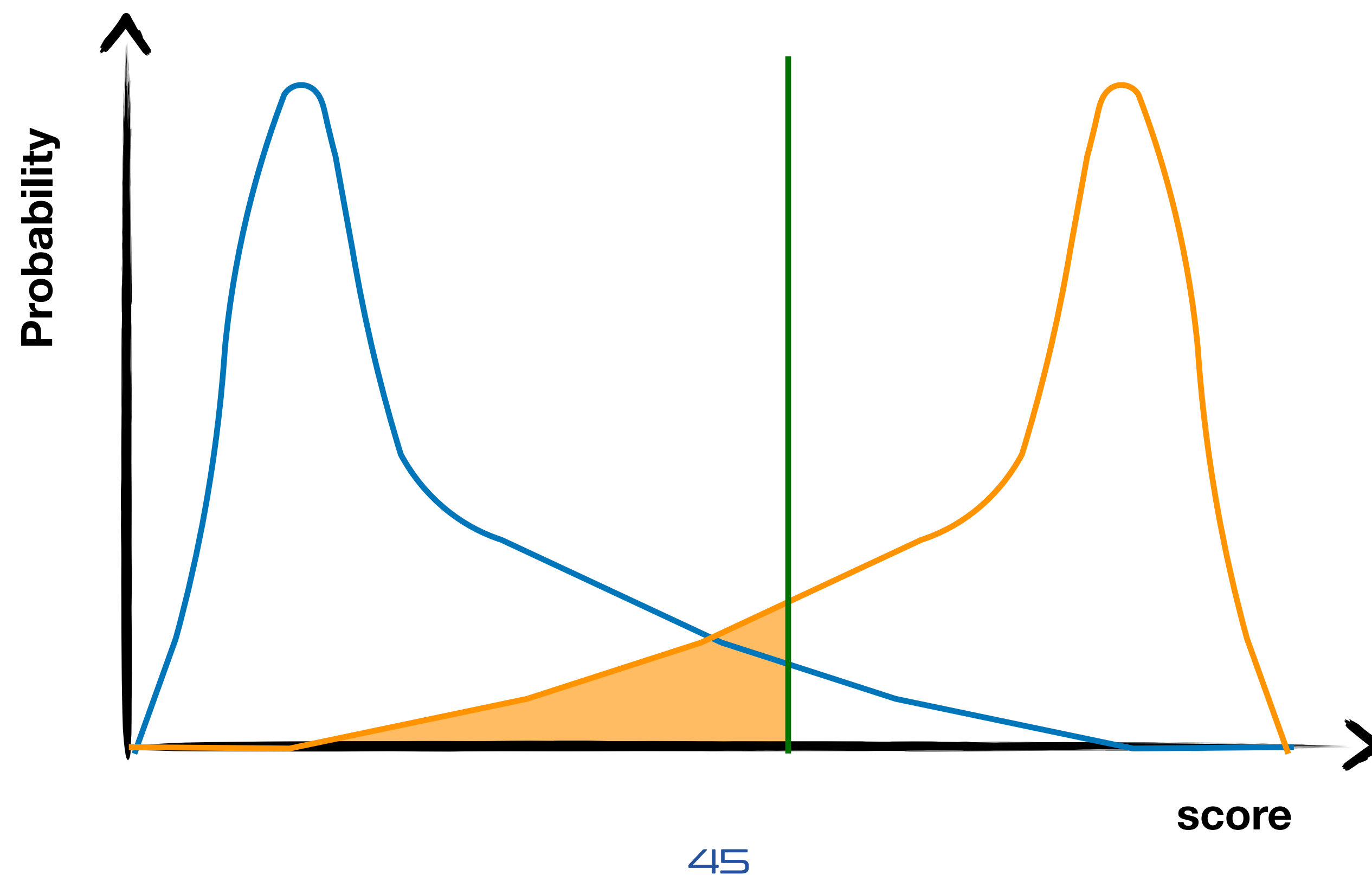
Classifier metrics

- A given threefold defines the following qualities
 - True-positives: Class-1 events above the threshold
 - True-negatives: Class-0 events below the threshold
 - False-positives: Class-0 events above the threshold
 - False-negatives: Class-1 events below the threshold



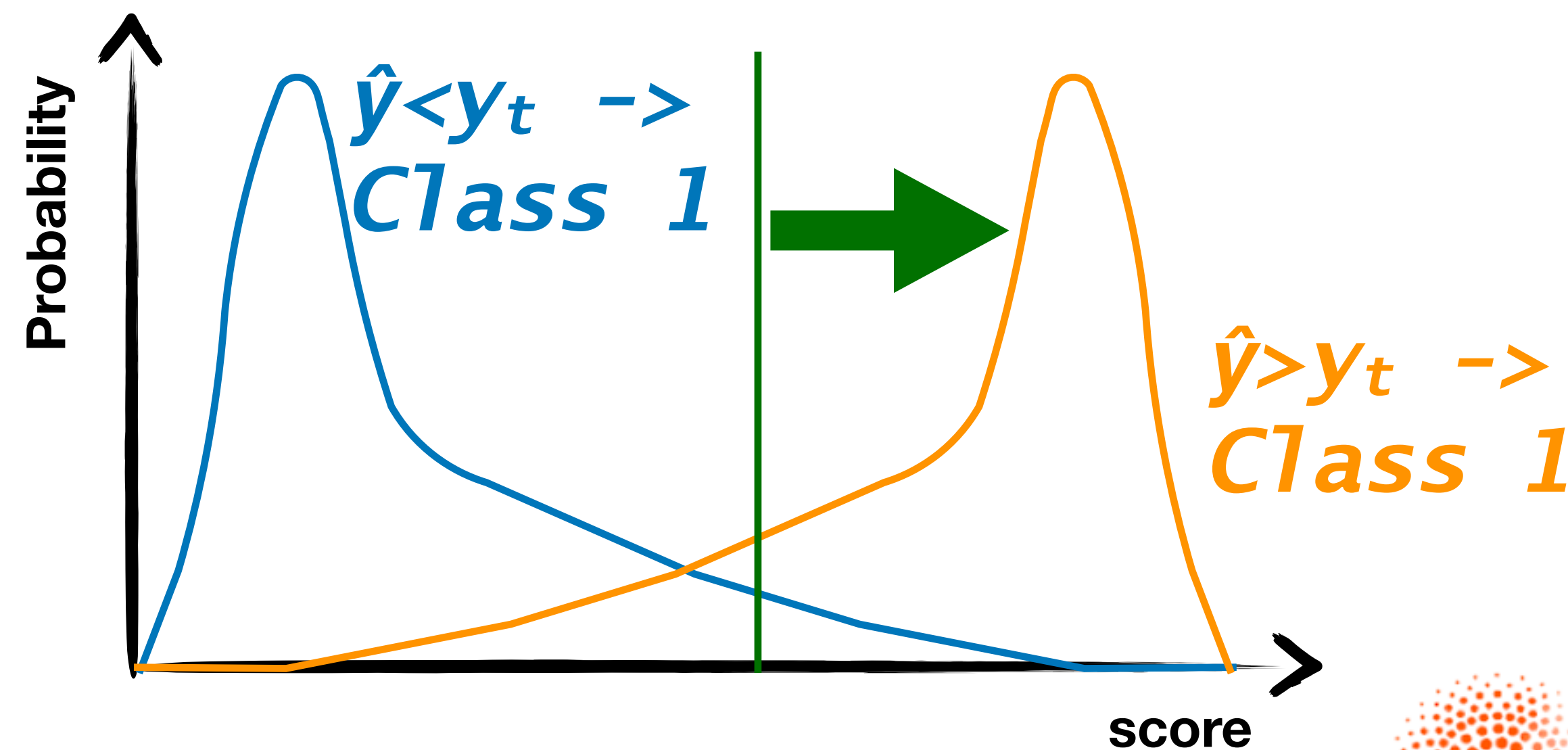
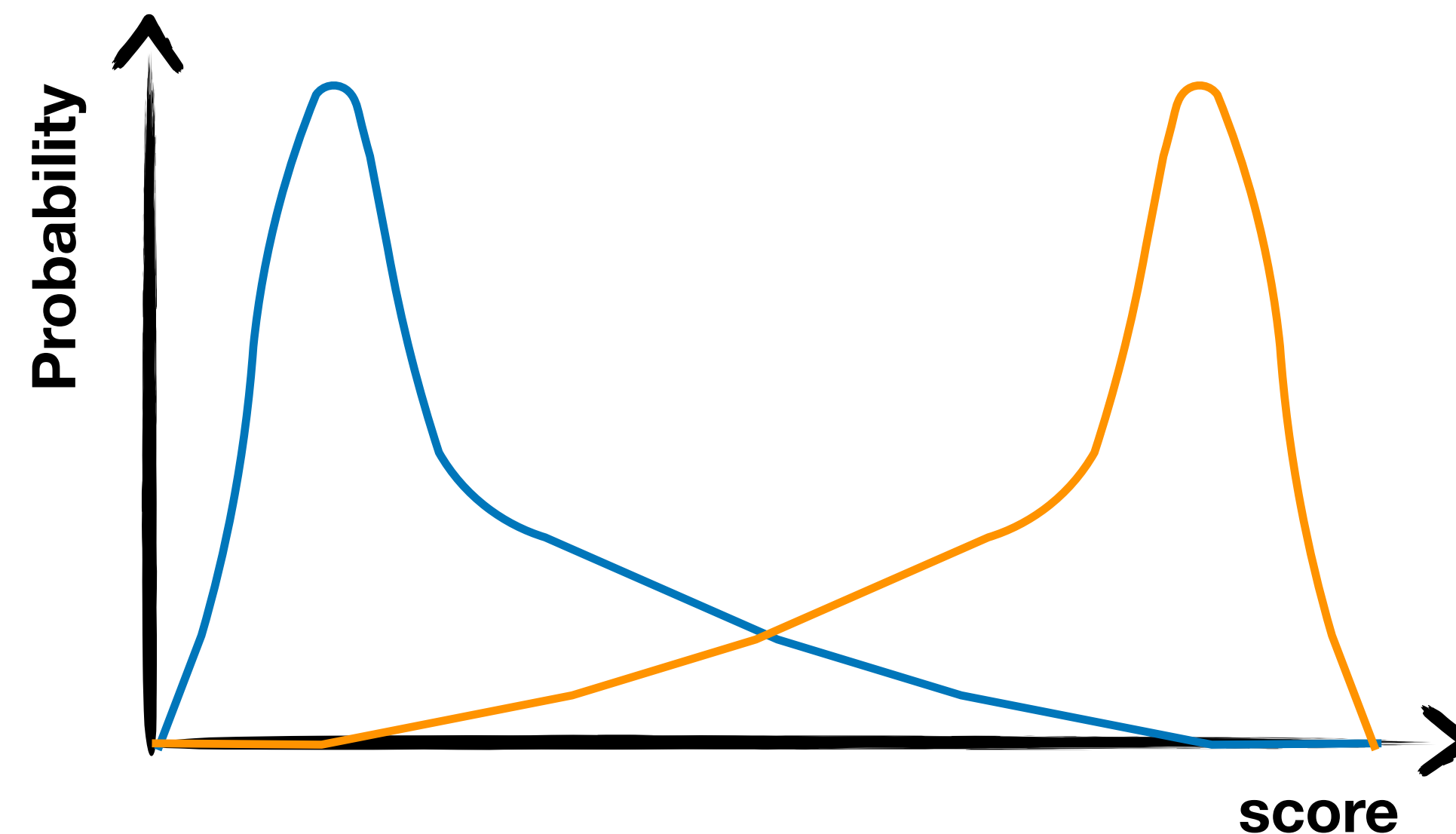
Classifier metrics

- A given threefold defines the following qualities
 - True-positives: Class-1 events above the threshold
 - True-negatives: Class-0 events below the threshold
 - False-positives: Class-0 events above the threshold
 - False-negatives: Class-1 events below the threshold



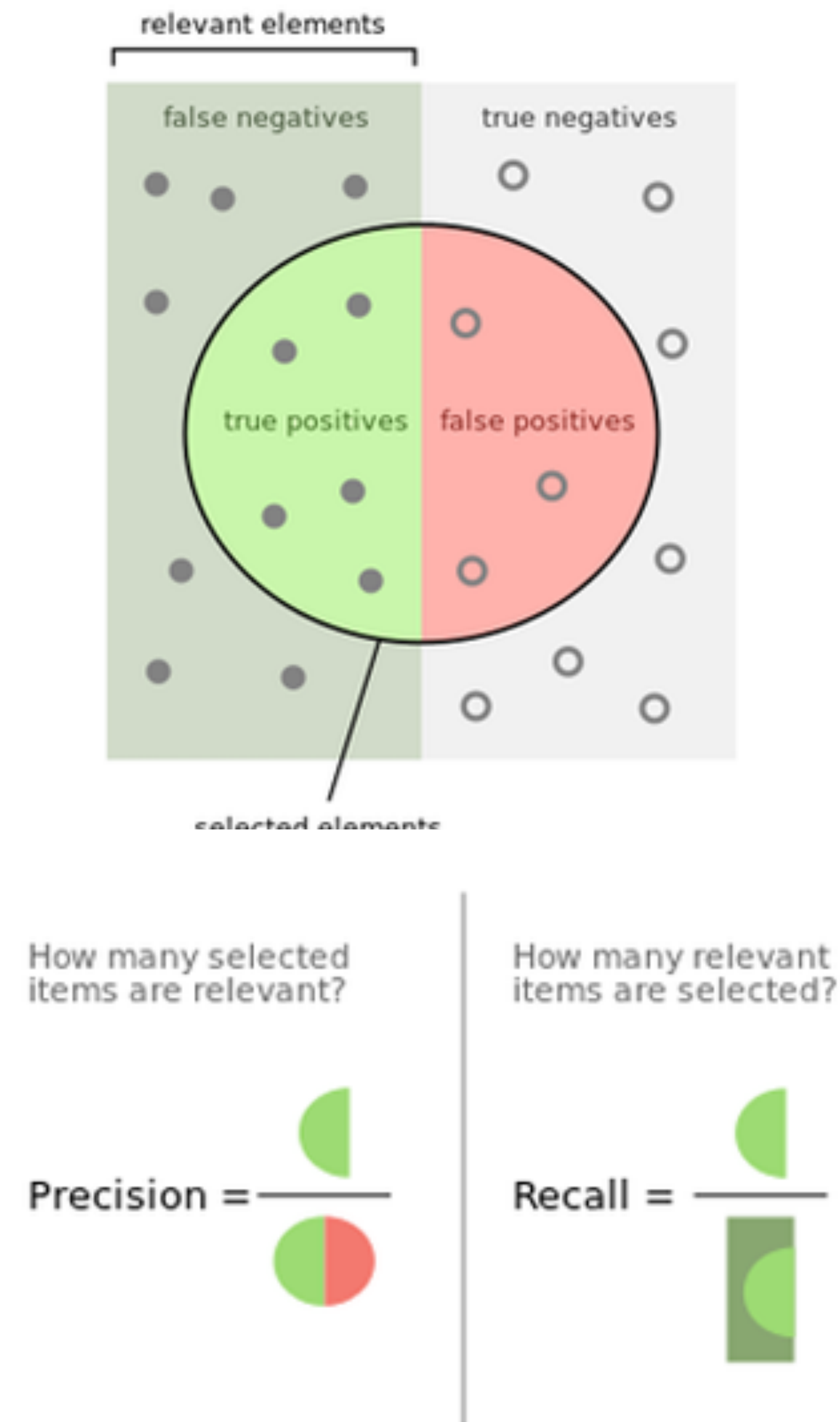
Classifier metrics

- Consider a binary classifier
- Its output $\hat{y}$ is a number in $[0,1]$
- If well trained, value should be close to 0 (1) for class-0 (class-1) examples
- One usually defines a threshold y_t such that:
 - $\hat{y} > y_t \rightarrow \text{Class 1}$
 - $\hat{y} < y_t \rightarrow \text{Class 0}$

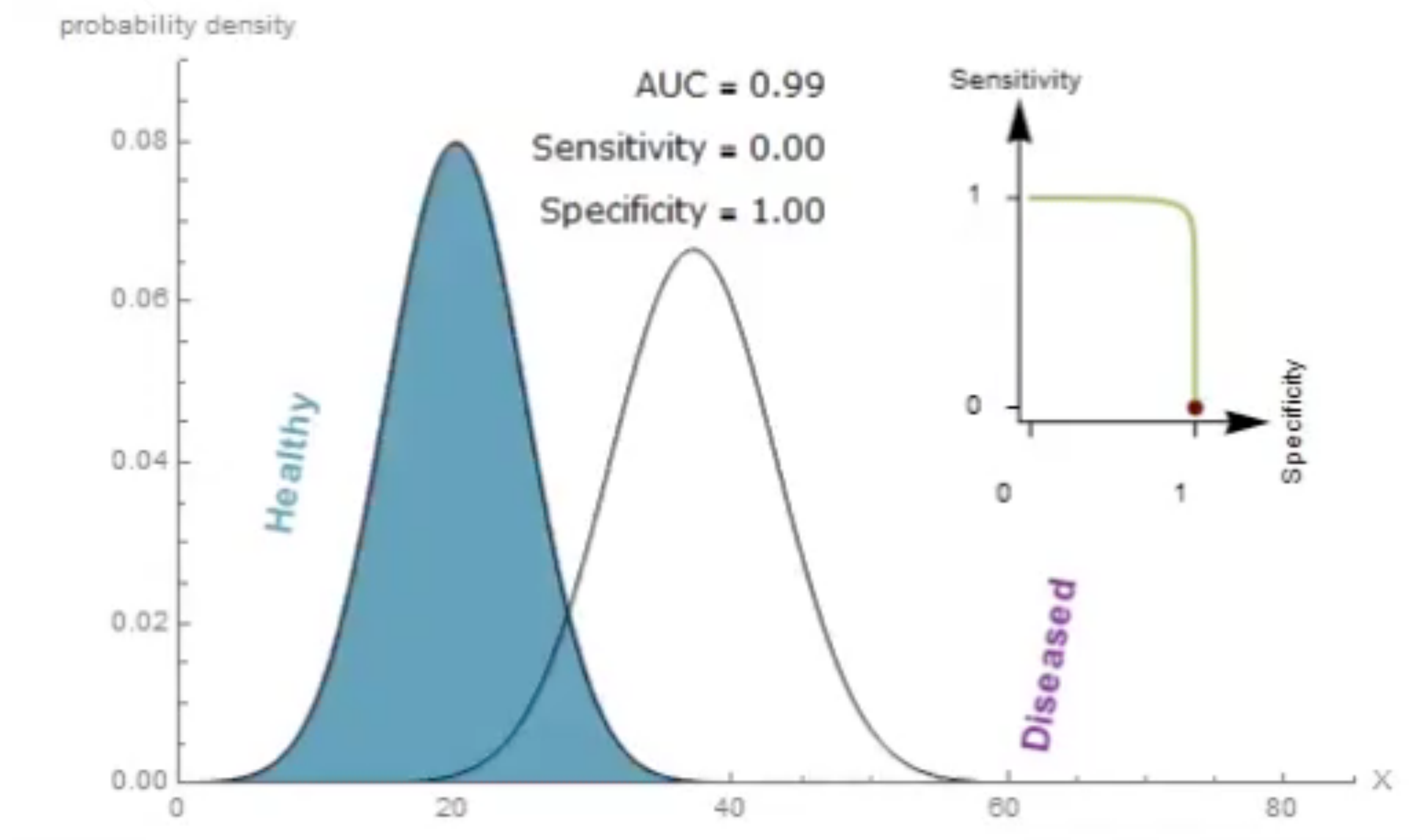


Classifier metrics

- Starting ingredients are true positive (TP) and true negative (TN) rates
- Accuracy: $(TP+TN)/Total$
 - The fraction of events correctly classified
- Sensitivity: $TP/(Total\ positive)$
 - AKA signal efficiency in HEP
- Specificity: $TN/(Total\ negative)$
 - AKA mistag rate in HEP



Receiver operating characteristic



- *Good choice of parameters can be done by hand, guided by your understanding of what each parameter does and how they interplay*

Hyperparameter	Increases capacity when...	Reason	Caveats
Number of hidden units	increased	Increasing the number of hidden units increases the representational capacity of the model.	Increasing the number of hidden units increases both the time and memory cost of essentially every operation on the model.
Learning rate	tuned optimally	An improper learning rate, whether too high or too low, results in a model with low effective capacity due to optimization failure.	
Convolution kernel width	increased	Increasing the kernel width increases the number of parameters in the model.	A wider kernel results in a narrower output dimension, reducing model capacity unless you use implicit zero padding to reduce this effect. Wider kernels require more memory for parameter storage and increase runtime, but a narrower output reduces memory cost.
Implicit zero padding	increased	Adding implicit zeros before convolution keeps the representation size large.	Increases time and memory cost of most operations.
Weight decay coefficient	decreased	Decreasing the weight decay coefficient frees the model parameters to become larger.	
Dropout rate	decreased	Dropping units less often gives the units more opportunities to “conspire” with each other to fit the training set.	

- Good choice of parameters can be done by large-scale calculations, repeating the training multiple times
- Scan the space with a regular grid or with a random scan
- Define the tuning as a minimisation process itself (e.g., with Bayesian optimization or other regression techniques)

